MICROSOFT CERTIFIED: AZURE AI CLOUD DEVELOPER ASSOCIATE

Exam AI-200 Study Guide

Developing AI Cloud Solutions on Azure

EXAM **AI-200**	SKILLS OUTLINE PUBLISHED **15 April 2026**
TIME TO COMPLETE **120 minutes, proctored**	PASSING SCORE **700 of 1000**
LEVEL **Intermediate · Associate**	RENEWAL **Free online, every 12 months**

Container Registry · App Service · Container Apps · AKS & KEDA · Cosmos DB · PostgreSQL & pgvector · Managed Redis · Service Bus · Event Grid · Functions · Key Vault · OpenTelemetry

Every Microsoft link in this guide carries the Microsoft Learn Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Language-locale segments have been removed and the ID is appended with `?` or `&` exactly as the Registered Member Handbook requires, so you can copy any link straight out of this document and share it.

Built from the official skills outline published on the Microsoft Learn AI-200 study guide page.

Table of contents

Chapter 0 — How to use this guide

0.1 What AI-200 actually certifies

0.2 The four skill areas and their weights

0.3 Exam logistics you should know before booking

0.4 A six-week study plan

0.5 How to read the callouts

Chapter 1 — The AI cloud solution blueprint

1.1 The reference shape

1.2 Identity: the answer to a surprising number of questions

1.3 SDK habits the exam expects

1.4 How the pieces are usually assembled

Chapter 2 — Containerized solutions, part 1: registry and App Service

2.1 Azure Container Registry

2.2 ACR Tasks — build in the cloud, not on your laptop

2.3 Deploying a container to App Service

2.4 Environment variables and secrets in App Service

2.5 Chapter 2 self-check

2.6 Learn more

Chapter 3 — Containerized solutions, part 2: Container Apps and AKS

3.1 Azure Container Apps: environments and revisions

3.2 Event-driven scaling with KEDA

3.3 Azure Kubernetes Service with manifest files

3.4 Monitoring and troubleshooting containers

3.5 Chapter 3 self-check

3.6 Learn more

Chapter 4 — AI solutions with Azure Cosmos DB for NoSQL

4.1 Connecting and querying with the SDK

4.2 Partitioning

4.3 Indexing policy

4.4 Consistency levels

4.5 Vector search for semantic retrieval

4.6 The change feed

4.7 Chapter 4 self-check

4.8 Learn more

Chapter 5 — AI solutions with Azure Database for PostgreSQL

5.1 Enabling the extensions

5.2 Schema design for RAG

5.3 Generating embeddings — in the app or in the database

5.4 Vector indexes and the operators

5.5 Similarity search with metadata filters (the RAG query)
5.6 Compute, memory, and connections
5.7 Chapter 5 self-check
5.8 Learn more

Chapter 6 — Azure Managed Redis in AI solutions

6.1 What Redis is for in an AI application
6.2 Data operations, expiration, and invalidation
6.3 Vector indexing and similarity search
6.4 Chapter 6 self-check
6.5 Learn more

Chapter 7 — Event- and message-based AI solutions

7.1 Choosing the right service
7.2 Azure Service Bus
7.3 Azure Event Grid
7.4 Putting them together
7.5 Chapter 7 self-check
7.6 Learn more

Chapter 8 — Azure Functions

8.1 The trigger and binding model
8.2 Hosting plans
8.3 Configuration and deployment
8.4 Durable Functions for long AI workflows
8.5 Chapter 8 self-check
8.6 Learn more

Chapter 9 — Secrets and configuration

9.1 Azure Key Vault
9.2 Azure App Configuration
9.3 Chapter 9 self-check
9.4 Learn more

Chapter 10 — Monitoring and troubleshooting

10.1 OpenTelemetry and distributed tracing
10.2 KQL for logs and metrics
10.3 A troubleshooting playbook
10.4 Chapter 10 self-check
10.5 Learn more

Chapter 11 — The hands-on lab track

Chapter 12 — Practice questions

Chapter 13 — Exam-day checklist and quick reference

13.1 The week before
13.2 The 60-second memory dump
13.3 Reading exam questions
13.4 Glossary

13.5 Where to go next

Appendix A — One-page decision tables

A.1 Which compute host?

A.2 Which data store?

A.3 Which messaging service?

A.4 Which security control?

A.5 Which telemetry answer?

Chapter 0 — How to use this guide

0.1 What AI-200 actually certifies

Passing **Exam AI-200: Developing AI Cloud Solutions on Azure** earns the Microsoft Certified: Azure AI Cloud Developer Associate credential. Microsoft's own description of the role:

> As a candidate for this Microsoft Certification, you're responsible for contributing to all phases of implementing AI solutions on Azure, with an emphasis on back-end services and components.

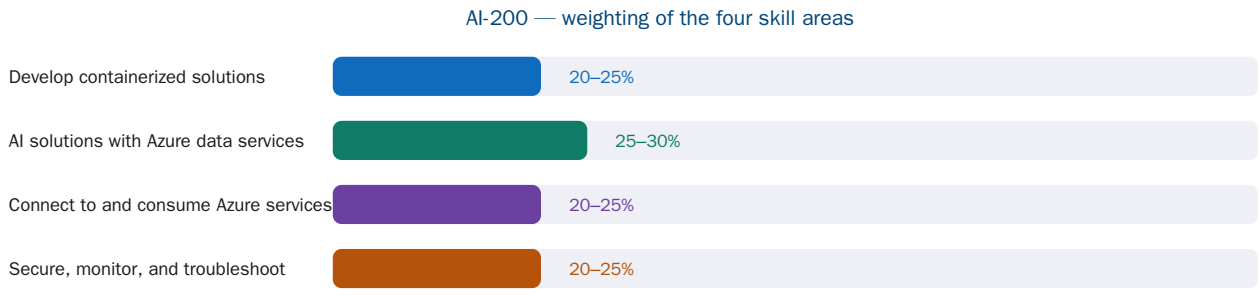
You are also responsible for the whole development lifecycle: requirements, design, development, deployment, security, and monitoring. From the official AI-200 study guide, you should be proficient in:

- Azure SDKs and third-party SDKs used in Azure
- Azure data management services
- Azure monitoring and troubleshooting
- Azure messaging and eventing
- Vector databases
- Python programming
- Implementing containerized applications on Azure

KEY TAKEAWAY

Read that list again: it is a **cloud developer** exam wearing an AI hat. You are not asked to train models or tune prompts — that is AI-300's territory. You are asked to build the back end that an AI application runs on: containers, databases with vector search, queues, functions, secrets, and telemetry.

0.2 The four skill areas and their weights



The four skill areas are close to evenly weighted, with the data services area slightly ahead.

Skill area	Weight	Chapters in this guide
Develop containerized solutions on Azure	20–25%	2, 3
Develop AI solutions by using Azure data management services	25–30%	4, 5, 6
Connect to and consume Azure services	20–25%	7, 8
Secure, monitor, and troubleshoot Azure solutions	20–25%	9, 10

0.3 Exam logistics you should know before booking

Item	Detail
Duration	**120 minutes** to complete the assessment; the appointment is longer — see exam duration and experience
Passing score	700 on a scale of 1000 — a scaled score, not a percentage. See exam scoring and score reports
Question count	Not published. Expect roughly 40–60 items, typically including a case study
Question types	Multiple choice, multi-select, drag-and-drop ordering, hot area, build-list, code completion, and possible interactive components
Languages	English, Arabic (Saudi Arabia), Chinese (Simplified), Chinese (Traditional), French, German, Indonesian, Italian, Japanese, Korean, Portuguese (Brazil), Russian, Spanish
Delivery	Proctored, online or at a Pearson VUE test centre
Retake	24 hours after a first failure, longer for later attempts — retake policy
Renewal	Free online assessment every 12 months — renew your certification
Accommodations	Request accommodations for extra time or assistive technology

WATCH OUT

At the time of writing, Microsoft has not published a free practice assessment for AI-200 the way it has for older exams. Check the AI-200 exam page before you book — if one has appeared since, take it twice. In the meantime, the self-check questions and Chapter 12 of this guide are your substitute, and the exam sandbox still lets you practise the interface.

0.4 A six-week study plan



Roughly 6–8 hours per week. This exam rewards muscle memory in the portal and the CLI, so do the labs.

Week	Focus	This guide	Microsoft Learn
1	Architecture, identity, SDK auth, containers on ACR and App Service	Ch. 1, 2	Implement container application hosting on Azure
2	Container Apps, KEDA, AKS	Ch. 3	Deploy and manage apps on Azure Container Apps · Deploy containers with AKS
3	Cosmos DB for NoSQL, vector search, change feed	Ch. 4	Develop AI solutions with Azure Cosmos DB for NoSQL
4	PostgreSQL + pgvector, Managed Redis	Ch. 5, 6	Build AI apps with Azure Database for PostgreSQL · Enhance AI solutions with Azure Managed Redis
5	Service Bus, Event Grid, Functions	Ch. 7, 8	Integrate backend services for AI solutions
6	Key Vault, App Configuration, OpenTelemetry, practice	Ch. 9, 10, 11, 12, 13	Manage application secrets and configuration · Observe and troubleshoot apps on Azure

0.5 How to read the callouts

Callout	What it means
Exam focus	Wording drawn straight from the official skills outline
Common exam trap	A distinction the exam deliberately tests; most wrong answers come from these
Know the difference	Two services that look alike; be able to choose between them under time pressure
Key takeaway	The one sentence to remember from the section
Hands-on lab	Do this in a real subscription. Reading alone will not get you to 700
Tip / Watch out	Practical advice and known gotchas

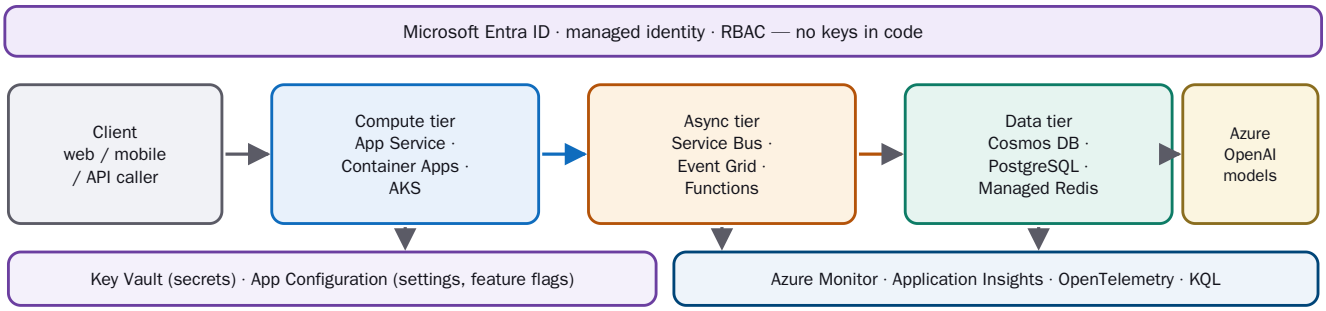
HANDS-ON LAB

Get an Azure free account (students get credit without a credit card) and create **one resource group** called `ai200-lab`. Everything in this guide can be built there, and you can delete the whole group afterwards to stop all charges.

Chapter 1 — The AI cloud solution blueprint

Every scenario in this exam is a variation on one architecture. Learn it once and the questions become recognisable.

1.1 The reference shape



The shape of nearly every AI-200 scenario: a compute tier, an async tier, a data tier with vectors, a model, and cross-cutting identity, config and telemetry.

Tier	Services you must know	Why it exists in an AI app
Compute	App Service, Container Apps, AKS	Hosts the API that orchestrates prompts, retrieval and tools
Async	Service Bus, Event Grid, Functions	Embedding, indexing and enrichment are slow — do them off the request path
Data	Cosmos DB, PostgreSQL, Managed Redis	Stores documents, embeddings, chat history and cached responses
Model	Azure OpenAI	Embeddings and completions
Cross-cutting	Key Vault, App Configuration, Azure Monitor	Secrets, settings, and the telemetry you will be asked to troubleshoot

1.2 Identity: the answer to a surprising number of questions

Almost every "how should this service authenticate to that service" question resolves to **managed identity**. See managed identities for Azure resources.

Option	When it is the right answer
System-assigned managed identity	One resource, one identity, deleted with the resource. The default good answer
User-assigned managed identity	Several resources share one identity, or the identity must outlive the resource (common with AKS workloads and deployment slots)
Service principal with a secret	Only when the caller is outside Azure and cannot use federated credentials
Connection string / account key	Legacy. If it appears as an option next to managed identity, it is usually the wrong answer

In code, use `DefaultAzureCredential` so the same code works locally (your developer login) and in Azure (managed identity):

```
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient
from azure.cosmos import CosmosClient

credential = DefaultAzureCredential()           # env vars → managed identity → Azure CLI
login → ...
secrets = SecretClient("https://kv-ai200.vault.azure.net/", credential)
cosmos  = CosmosClient("https://cosmos-ai200.documents.azure.com:443/", credential)
```

Reference: authentication overview for the Azure SDK for Python.

> **COMMON EXAM TRAP**
> A managed identity by itself grants **nothing**. You must also assign a **role** on the target resource — `Key Vault Secrets User`, `Cosmos DB Built-in Data Reader`, `Azure Service Bus Data Sender`, and so on. "The app has a managed identity but still gets 403" is a role-assignment question, not an identity question.

1.3 SDK habits the exam expects

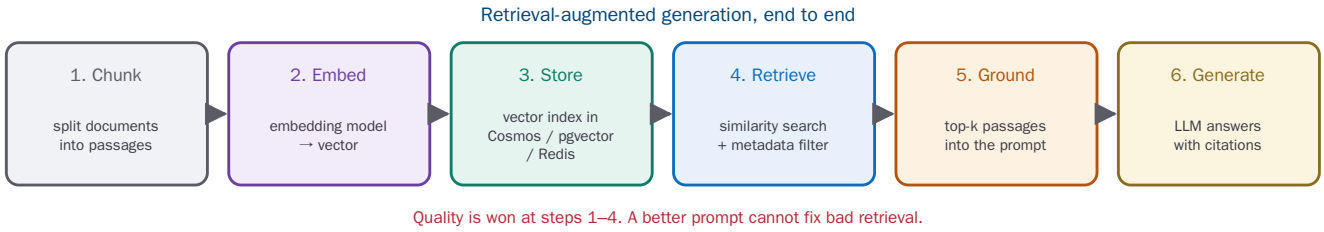
- **Create clients once and reuse them.** `CosmosClient`, `ServiceBusClient`, `BlobServiceClient` and HTTP clients are designed to be long-lived singletons; creating one per request exhausts sockets and destroys latency.
- **Use async clients** (`azure.cosmos.aio`, `azure.servicebus.aio`) in web APIs that fan out to several services.
- **Let the SDK retry.** The default retry policies already handle 429 and transient 5xx with exponential backoff; layering your own loop on top usually makes throttling worse.
- **Never log the payload of a prompt or a secret.** Telemetry is the most common accidental data leak in AI apps.

1.4 How the pieces are usually assembled

A typical exam scenario, and the services it maps to:

> *"A chat application must answer questions about internal PDFs. Documents are uploaded to storage. Answers must cite sources. Uploads must not slow the chat API. Costs must stay low when nobody is using it."*

Requirement	Service and why
Documents uploaded	Blob Storage + Event Grid event on blob created
Chunk and embed without blocking	Azure Functions triggered by the event, or a Service Bus queue for retry control
Store embeddings and metadata	Cosmos DB vector search or pgvector
Cache repeated answers	Azure Managed Redis
Host the chat API cheaply	Container Apps with `minReplicas = 0`
Keep keys out of code	Key Vault + managed identity
Diagnose slow answers	OpenTelemetry traces in Application Insights

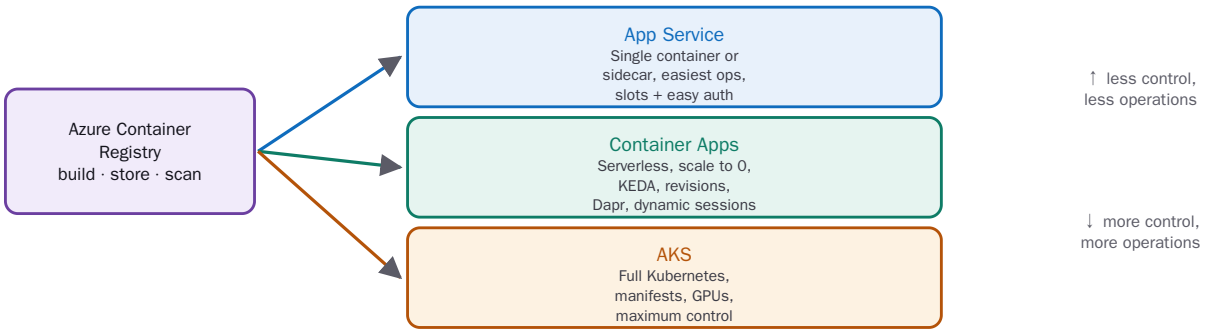


The retrieval pipeline behind that scenario. Most AI-200 data questions are about steps 2–4.

> **KEY TAKEAWAY**
>
> If you can draw this diagram from memory and name the Azure service at each step, you already understand the spine of the exam. The rest of this guide is the detail of each box.

Chapter 2 — Containerized solutions, part 1: registry and App Service

> **EXAM FOCUS**
>
> **Implement container application hosting** — build, store, version, and manage container images by using Azure Container Registry · build and run images by using Azure Container Registry Tasks · deploy containers to Azure App Service, including configuring App Service to supply environment variables and secrets.



One registry, three hosting targets. The exam tests whether you pick the cheapest host that still meets the requirement.

2.1 Azure Container Registry

Azure Container Registry (ACR) is a managed, private Docker registry.

SKU	What it adds
Basic	Lowest cost, same API, small included storage and throughput. Dev and test
Standard	More storage and throughput. Most production workloads
Premium	Geo-replication, private link, content trust/image signing, customer-managed keys, scope-mapped tokens, highest throughput

Know these facts:

- Registry names are **globally unique** and the login server is `<name>.azurecr.io`.
- **Admin user is disabled by default** and should stay that way — it is a shared username/password. Authenticate with Entra ID (`az acr login`), a managed identity, or a service principal instead. See registry authentication.
- **Geo-replication is Premium only** and gives a single login server with regional endpoints — the right answer when clusters in several regions pull the same image.
- Use **immutable, unique tags** (`app:1.4.2`, `app:<git-sha>`) for deployments and **stable tags** (`base:8.0`) only for base images. See image tag best practices.

2.2 ACR Tasks — build in the cloud, not on your laptop

ACR Tasks build container images **inside Azure**, so you do not need Docker installed and the build happens close to the registry.

```
# Quick task: build and push in one command, no local Docker daemon
az acr build --registry acrai200 --image chatapi:1.0.0 --file Dockerfile .

# Automatically triggered task: rebuild on every commit AND when the base image is patched
az acr task create \
  --registry acrai200 --name chatapi-build \
  --image chatapi:{{.Run.ID}} \
  --context https://github.com/contoso/chatapi.git \
  --file Dockerfile --git-access-token $PAT \
  --base-image-trigger-enabled true
```

Three task types to recognise:

Type	Trigger
Quick task	You run `az acr build` on demand
Automatically triggered task	Source-code commit, **base image update**, or a schedule
Multi-step task	A YAML file with build/push/test steps and dependencies

> **COMMON EXAM TRAP**
>
> **Base image update triggers** are the classic ACR Tasks exam answer. "When a security patch is released for the base image, all dependent images must be rebuilt automatically, with no pipeline changes" = an ACR task with `--base-image-trigger-enabled`. A GitHub Actions schedule is the wrong answer because it rebuilds on a clock, not on the actual base image event.

2.3 Deploying a container to App Service

App Service runs a single container (or a container plus sidecars) with the least operational effort of the three hosts. See run a custom container.

```
az webapp create --resource-group ai200-lab --plan plan-ai200 \
  --name app-chatapi --container-image-name acrai200.azurecr.io/chatapi:1.0.0

# Pull from ACR with the app's managed identity instead of registry credentials
az webapp identity assign --resource-group ai200-lab --name app-chatapi
az webapp config set --resource-group ai200-lab --name app-chatapi \
  --generic-configurations '{"acrUseManagedIdentityCreds": true}'
```

2.4 Environment variables and secrets in App Service

This is called out explicitly in the skills outline, so know the mechanics.

App settings become environment variables in the container. They are encrypted at rest and never appear in source control:

```
az webapp config appsettings set --resource-group ai200-lab --name app-chatapi \
  --settings AZURE_OPENAI_ENDPOINT="https://oai-ai200.openai.azure.com/" \
             EMBEDDING_MODEL="text-embedding-3-small"
```

Key Vault references keep the secret in Key Vault and let App Service resolve it at runtime — the app still just reads an environment variable:

```
COSMOS_KEY = @Microsoft.KeyVault(SecretUri=https://kv-ai200.vault.azure.net/secrets/cosmos-key/)
```

The app's managed identity needs the `Key Vault Secrets User` role. Reference: Key Vault references for App Service.

Deployment slots give you a warm staging copy and an instant swap:

Concept	Behaviour
Slot	A live app with its own hostname sharing the plan
Swap	Exchanges the running instances — near-zero downtime, instantly reversible
Slot setting ("deployment slot setting" checkbox)	A setting that **stays with the slot** and does not travel during a swap
Non-slot setting	Travels with the app during a swap

Reference: deployment slots and configure common settings.

> **COMMON EXAM TRAP**
> The slot-setting checkbox is a favourite. If staging must point at a test Cosmos DB and production at the production one, mark the connection setting as a **slot setting** — otherwise the swap carries production's config into staging and vice versa.

> **TIP**
> App Service **sidecar containers** are the low-effort way to add a local model, a vector cache, or an OpenTelemetry collector next to an existing web app without moving to Kubernetes. See the Learn module run sidecar-enabled AI applications on Azure App Service.

2.5 Chapter 2 self-check

1. A base image CVE is patched. Every derived image must be rebuilt automatically. What do you configure?
2. Which ACR SKU is required for a single image pulled by clusters in three regions with local latency?

3. An App Service container must read a Cosmos DB key without the key ever appearing in configuration exports. What do you use?
4. After a swap, staging is suddenly pointing at the production database. What was misconfigured?
5. How should App Service authenticate to ACR in a production deployment?

> **Answers.** 1. An ACR task with a base image update trigger. 2. Premium (geo-replication). 3. A Key Vault reference in an app setting, resolved with the app's managed identity. 4. The connection setting was not marked as a **deployment slot setting**. 5. With the web app's managed identity and `acrUseManagedIdentityCreds`, not the admin user.

2.6 Learn more

- Introduction to Azure Container Registry · ACR Tasks · authentication · registry concepts
- App Service overview · custom containers · managed identity · Key Vault references · deployment slots
- Training: Implement container application hosting on Azure · Store and manage containers in ACR · Deploy containers to App Service

---

Chapter 3 — Containerized solutions, part 2: Container Apps and AKS

> **EXAM FOCUS**
> **Implement container-orchestrated solutions** — deploy applications to Azure Container Apps, including environment configuration and revision management · implement event-driven scaling by using Kubernetes Event-driven Autoscaling (KEDA) in Container Apps · deploy and manage applications to Azure Kubernetes Service (AKS) by using manifest files · monitor and troubleshoot solutions on AKS and Container Apps by inspecting logs, events, and end-to-end connectivity.

3.1 Azure Container Apps: environments and revisions

Azure Container Apps is serverless containers: Kubernetes underneath, none of it exposed.

An **environment** is the security and networking boundary. Apps in the same environment share a virtual network and a Log Analytics workspace, and can reach each other by name over internal ingress. Put applications that talk to each other in the **same** environment; put unrelated workloads in different ones.

A **revision** is an immutable snapshot of a container app version.

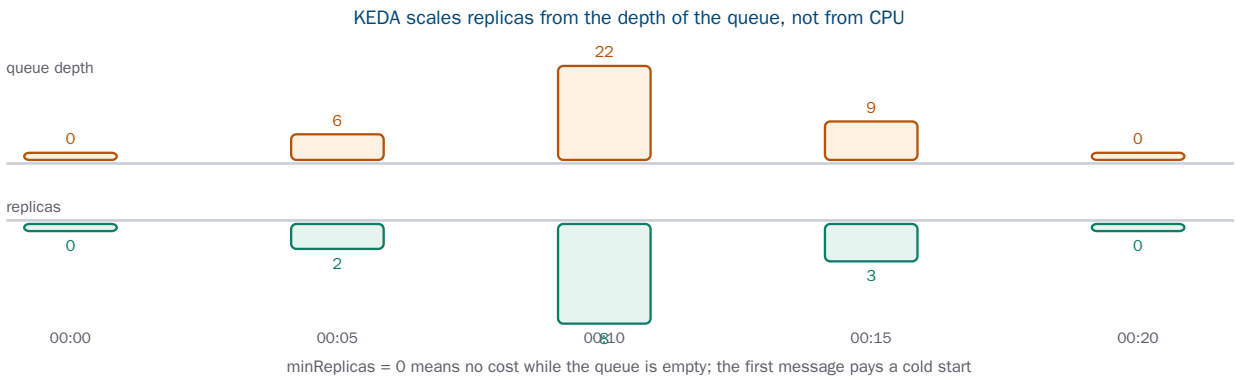
Mode	Behaviour
Single revision (default)	A new revision replaces the old one; 100% of traffic moves
Multiple revisions	Several revisions run at once and you **split traffic by weight** — blue/green and canary releases

```
az containerapp update --name chatapi --resource-group ai200-lab \
  --image acrai200.azurecr.io/chatapi:1.1.0 --revision-suffix v11

az containerapp ingress traffic set --name chatapi --resource-group ai200-lab \
  --revision-weight chatapi--v10=80 chatapi--v11=20      # 20% canary
```

Changing a **revision-scope** property (image, environment variables, compute) creates a new revision. Changing an **application-scope** property (ingress rules, secret values, traffic weights) does not.

3.2 Event-driven scaling with KEDA



KEDA scales on the backlog, not the CPU. This is the property the exam tests.

Container Apps has KEDA built in. A scale rule can be HTTP concurrency, TCP, CPU/memory, or any KEDA scaler — Service Bus queue length, Event Hubs lag, Kafka, Azure Queue Storage, cron, and more.

```
az containerapp update --name embedder --resource-group ai200-lab \
  --min-replicas 0 --max-replicas 30 \
  --scale-rule-name sb-queue \
  --scale-rule-type azure-servicebus \
  --scale-rule-metadata queueName=embed-jobs messageCount=20 namespace=sb-ai200 \
  --scale-rule-auth connection=sb-connection
```

Property	Meaning
`minReplicas: 0`	**Scale to zero** — no compute charge when idle, at the cost of a cold start on the next event
`minReplicas: 1+`	Always-warm, no cold start, always billed
`messageCount`	Target messages per replica; 100 messages with `messageCount=20` asks for 5 replicas
`maxReplicas`	The ceiling that protects your downstream database and your budget

COMMON EXAM TRAP

CPU and memory scale rules cannot scale to zero. Only HTTP, TCP, and custom (KEDA) rules can. A question that says "must cost nothing when idle *and* scale on queue depth" requires a KEDA queue rule with `minReplicas: 0`; answering "CPU-based autoscale" is wrong on both counts.

TIP

Two Container Apps features exist specifically for AI workloads: **dynamic sessions** run LLM-generated code in a Hyper-V isolated sandbox (the safe answer for "execute untrusted model-generated Python"), and **serverless GPUs** run inference without managing a GPU node pool.

3.3 Azure Kubernetes Service with manifest files

AKS is the answer when you need real Kubernetes: custom operators, GPU node pools you control, service meshes, or an existing Kubernetes estate.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: chatapi
spec:
  replicas: 3
  selector:
    matchLabels: { app: chatapi }
  template:
    metadata:
      labels: { app: chatapi, azure.workload.identity/use: "true" }
    spec:
      serviceAccountName: chatapi-sa           # federated to a user-assigned managed
identity
      containers:
        - name: chatapi
          image: acrai200.azurecr.io/chatapi:1.1.0
          ports: [{ containerPort: 8000 }]
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { cpu: "1",    memory: "1Gi" }
          readinessProbe:
            httpGet: { path: /healthz, port: 8000 }
            initialDelaySeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: chatapi
spec:
  type: ClusterIP
  selector: { app: chatapi }
  ports: [{ port: 80, targetPort: 8000 }]
```

```
az aks get-credentials --resource-group ai200-lab --name aks-ai200
kubectl apply -f deployment.yaml
kubectl get pods -o wide
kubectl rollout status deployment/chatapi
```

Scaling in AKS comes in three layers — know which one a question is describing:

Layer	Scales	Mechanism
HPA	Pods	CPU/memory or custom metrics
KEDA add-on	Pods	External events (queue length, stream lag) — see KEDA on AKS
Cluster autoscaler	Nodes	Adds nodes when pods cannot be scheduled

Reference: scaling concepts and configure scaling. For identity, use workload identity, which federates a Kubernetes service account to a managed identity — the modern replacement for pod-managed identity.

3.4 Monitoring and troubleshooting containers

The skills outline says "inspecting logs, events, and end-to-end connectivity", so expect symptom-to-cause questions.

Symptom	Almost always means	First command / place to look
`ImagePullBackOff`	Wrong image name/tag, or no pull permission on ACR	`kubectl describe pod` → Events; check `AcrPull` role on the kubelet identity
`CrashLoopBackOff`	The process exits immediately — bad config or missing env var	`kubectl logs <pod> --previous`
`OOMKilled`	Container exceeded its memory **limit**	`kubectl describe pod`; raise limits or fix the leak
`Pending` forever	No node has room, or a node selector/taint does not match	`kubectl describe pod` → Events; cluster autoscaler status
502/503 through ingress	Readiness probe failing, or wrong `targetPort`	Probe config, `kubectl get endpoints`
Container App stuck on old code	Traffic still weighted to the previous revision	`az containerapp revision list`, then set traffic
Container App exits instantly	The app is not listening on the configured `targetPort`	Ingress config and console logs

Where the logs live:

- **AKS** — Container insights sends `ContainerLogV2` to Log Analytics; `kubectl logs`, `kubectl get events --sort-by=.lastTimestamp`, and AKS diagnostics for cluster-level problems. Deeper triage: AKS troubleshooting documentation.
- **Container Apps** — log monitoring writes **console logs** (your stdout/stderr) and **system logs** (platform events) to Log Analytics, queryable with KQL. See also observability.

```
// Container Apps: errors from one app in the last hour
ContainerAppConsoleLogs_CL
| where ContainerAppName_s == "chatapi" and TimeGenerated > ago(1h)
| where Log_s contains "ERROR"
| project TimeGenerated, RevisionName_s, Log_s
| order by TimeGenerated desc
```

> **KNOW THE DIFFERENCE**
>
> **Container Apps vs AKS.** Container Apps when the requirement is "minimise operational overhead", "scale to zero", "HTTP or queue-driven microservices". AKS when the requirement mentions "Kubernetes manifests we already have", "custom CRDs/operators", "specific GPU SKUs and node pools", or "service mesh". If both would work, the exam's answer is Container Apps — it is the lower-effort option.

3.5 Chapter 3 self-check

1. A worker must cost nothing overnight and scale on Service Bus backlog. Configure what?
2. You need 10% of traffic on a new version for an hour. What must be enabled first?
3. A pod shows `ImagePullBackOff` right after a deployment to a new cluster. Most likely cause?
4. LLM-generated Python must be executed safely per user session. Which feature?
5. Which AKS component adds **nodes** when pods cannot be scheduled?

> **Answers.** 1. A KEDA `azure-servicebus` scale rule with `minReplicas: 0`. 2. Multiple-revision mode, then traffic weights. 3. The cluster's kubelet identity lacks `AcrPull` on the registry (or the tag is wrong). 4. Container Apps dynamic sessions. 5. The cluster autoscaler (HPA and KEDA scale pods, not nodes).

3.6 Learn more

- Container Apps overview · environments · revisions · scaling · managed identity · ingress · dynamic sessions · serverless GPUs
- AKS overview · clusters and workloads · scaling · KEDA add-on · workload identity · monitor AKS · networking concepts
- Training: Deploy and manage apps on Azure Container Apps · Scale containers in Container Apps · Run AI-generated code in dynamic sessions · Deploy containers by using AKS

Chapter 4 — AI solutions with Azure Cosmos DB for NoSQL

EXAM FOCUS

Develop AI solutions by using Azure Cosmos DB for NoSQL — connect to Azure Cosmos DB for NoSQL by using the SDK and run queries · optimize query performance and Request Units (RUs) consumption by using indexing policies and consistency levels · store and retrieve embeddings and execute vector similarity search for semantic retrieval · implement a change feed processor to detect and handle new or updated items.

4.1 Connecting and querying with the SDK

```
from azure.cosmos.aio import CosmosClient
from azure.identity.aio import DefaultAzureCredential

client    = CosmosClient(URL, credential=DefaultAzureCredential())   # singleton, reuse it
container = client.get_database_client("knowledge").get_container_client("chunks")

# Parameterised query, scoped to one logical partition
query = "SELECT c.id, c.title, c.text FROM c WHERE c.tenantId = @t AND c.category = @cat"
items = container.query_items(
    query=query,
    parameters=[{"name": "@t", "value": tenant}, {"name": "@cat", "value": "policy"}],
    partition_key=tenant,            # single-partition query: much cheaper
)
async for item in items:
    ...
```

Cost facts worth memorising:

Operation	Typical cost
Point read (`read_item` with id + partition key)	~**1 RU** for a 1 KB item — always the cheapest way to fetch a known document
Single-partition query	Cheap, scales with the number of items examined
Cross-partition query	Fans out to every physical partition; cost grows with the container
Write of a 1 KB item	~5 RU, more with many indexed properties

COMMON EXAM TRAP

`SELECT *` on a cross-partition query is the standard "why is this expensive" answer. Fix it by supplying the **partition key**, projecting only the fields you need, and removing unused properties from the indexing policy.

4.2 Partitioning

Choose a partition key that is **high cardinality**, **evenly distributed**, and **present in most queries**. A logical partition has a **20 GB** limit, so a key such as `tenantId` breaks when one tenant grows large — hierarchical partition keys (for example `/tenantId` then `/documentId`) let you subdivide without redesigning.

Symptoms of a bad key: one partition far hotter than the others ("hot partition"), 429s concentrated on one range, and RU charges that do not fall when you add throughput.

4.3 Indexing policy

Cosmos DB indexes **every property by default**. That makes reads flexible and writes expensive.

```
{
  "indexingMode": "consistent",
  "includedPaths": [{ "path": "/tenantId/?" }, { "path": "/category/?" }, { "path":
"/updatedAt/?" }],
  "excludedPaths": [{ "path": "/embedding/*" }, { "path": "/rawText/?" }, { "path": "/*" }],
  "compositeIndexes": [[{ "path": "/category", "order": "ascending" },
                        { "path": "/updatedAt", "order": "descending" }]],
  "vectorIndexes": [{ "path": "/embedding", "type": "diskANN" }]
}
```

- **Exclude large, never-filtered properties** — raw text and the embedding array itself — to cut write RUs sharply.
- Add a **composite index** for every `ORDER BY` over two or more properties, otherwise the query fails or is expensive.
- `indexingMode: none` is for pure key-value workloads only.

Reference: indexing policies and optimize cost of reads and writes.

4.4 Consistency levels



Choosing a consistency level is a deliberate trade of RU cost and latency against read guarantees.

The account default is **Session**, and it is the right answer for most applications: a client reads its own writes without paying for global linearisability. You can **relax** consistency per request (for example to Eventual for a cheap recommendation read) but never strengthen it beyond the account setting. See consistency levels.

KNOW THE DIFFERENCE

Strong vs Bounded staleness vs Session. Strong = every reader sees the latest committed write, highest RU and latency, and restricts multi-region write topologies. Bounded staleness = lag capped by *k* versions or *t* seconds. Session = consistency guaranteed *within a client session*. Pick Session unless the scenario says "must never read a stale value anywhere in the world".

4.5 Vector search for semantic retrieval

Enable the vector search capability on the account, define a **vector embedding policy** when you create the container, and add a **vector index**.

```
// container definition (excerpt)
"vectorEmbeddingPolicy": {
  "vectorEmbeddings": [{
    "path": "/embedding",
    "dataType": "float32",
    "distanceFunction": "cosine",
    "dimensions": 1536
  }]
}
```

Vector index type	Behaviour
`flat`	Exact search, stores vectors in the index. Small collections only
`quantizedFlat`	Compressed vectors, exact-ish and much smaller — good middle ground
`diskANN`	Approximate nearest neighbour, designed for large collections and low latency

Query with `VectorDistance()`, and always filter first:

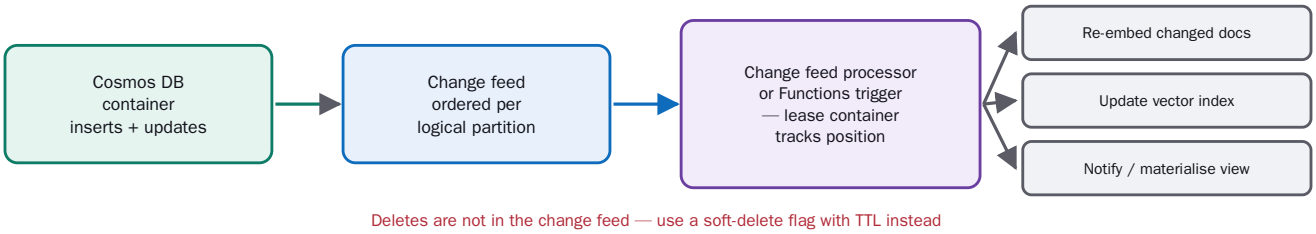
```
SELECT TOP 5 c.id, c.title, c.text,
       VectorDistance(c.embedding, @queryVector) AS similarityScore
FROM c
WHERE c.tenantId = @tenantId AND c.language = 'en'
ORDER BY VectorDistance(c.embedding, @queryVector)
```

COMMON EXAM TRAP

The embedding **dimensions in the policy must match the model exactly** (`text-embedding-3-small` = 1536, `text-embedding-3-large` = 3072). They cannot be changed after the container is created, and the distance function must match how the embeddings were generated — cosine for OpenAI embeddings. "We switched embedding models and search broke" means a new container, not a config tweak.

Reference: vector search in Azure Cosmos DB for NoSQL and Azure Cosmos DB as a vector database.

4.6 The change feed



The change feed is how an AI system keeps its vector index in step with its source of truth.

The change feed is a persistent, ordered record of inserts and updates, ordered **within each logical partition**. Two ways to consume it:

Approach	Use it when
Change feed processor in your own host	You need control over batching, parallelism and checkpointing
Azure Functions Cosmos DB trigger	You want the platform to host and scale the processor for you

```
# Azure Functions (Python v2 model): re-embed every changed document
import azure.functions as func
app = func.FunctionApp()

@app.cosmos_db_trigger(arg_name="docs", connection="COSMOS_CONN",
                       database_name="knowledge", container_name="chunks",
                       lease_container_name="leases",
create_lease_container_if_not_exists=True)
def reembed(docs: func.DocumentList):
    for doc in docs:
        vector = embed(doc["text"])
        upsert_vector(doc["id"], vector)
```

Facts the exam leans on:

- A **lease container** stores the processing position; several instances coordinate through it and each takes a share of the partitions.
- The default (latest version) mode delivers the **most recent version** of a changed item — if an item changes three times between reads you see it once.
- **Deletes do not appear** in the default mode. Use a soft-delete flag plus TTL, or the all-versions-and-deletes mode.
- Processing is **at-least-once**, so the handler must be idempotent.

4.7 Chapter 4 self-check

1. Which read operation costs about 1 RU?
2. Writes are expensive on a container that stores 1536-dimension embeddings. What is the first fix?
3. A query orders by `category` then `updatedAt` and fails. What is missing?

4. You need to re-embed documents when they change, with no host to manage. What do you build?
5. Documents deleted in Cosmos DB remain searchable in the vector index. Why?

> **Answers.** 1. A point read (`read_item`) with the id and partition key. 2. Exclude `/embedding/*` (and other large, unfiltered paths) from the indexing policy. 3. A composite index on those two paths. 4. An Azure Functions Cosmos DB trigger with a lease container. 5. The default change feed does not emit deletes — use soft deletes with TTL, or all-versions-and-deletes mode.

4.8 Learn more

- Azure Cosmos DB for NoSQL · getting started with queries · request units · partitioning · indexing policies · consistency levels
- Vector search · vector database · change feed · change feed processor · get started with Python
- Training: Develop AI solutions with Azure Cosmos DB for NoSQL · Implement vector search · Optimize query performance · Handle events with the change feed

---

Chapter 5 — AI solutions with Azure Database for PostgreSQL

> **EXAM FOCUS**
>
> **Develop AI solutions by using Azure Database for PostgreSQL** — connect and query by using SDKs · model schemas and implement indexing strategies, including designing tables and choosing appropriate data types · implement indexing strategies, including optimizing query latency and reducing pgvector compute overhead · configure compute, memory, and storage resources to support vector workloads · run vector similarity search, including storing embeddings, semantic retrieval, and implementing RAG patterns by using metadata filters · implement connection optimization to improve throughput and minimize latency.

5.1 Enabling the extensions

On Azure Database for PostgreSQL flexible server, extensions must be **allow-listed on the server first**, then created in the database. This two-step requirement is exam material.

```
az postgres flexible-server parameter set \
  --resource-group ai200-lab --server-name pg-ai200 \
  --name azure.extensions --value vector,azure_ai,pg_diskann
```

```
CREATE EXTENSION IF NOT EXISTS vector;     -- pgvector: the vector data type and operators
CREATE EXTENSION IF NOT EXISTS azure_ai;   -- call Azure OpenAI from SQL
```

Reference: how to use pgvector and generative AI with Azure Database for PostgreSQL.

5.2 Schema design for RAG

```
CREATE TABLE doc_chunks (
    id           bigserial PRIMARY KEY,
    tenant_id    uuid        NOT NULL,
    doc_id       uuid        NOT NULL,
    chunk_no     int         NOT NULL,
    language     text        NOT NULL,
    updated_at   timestamptz NOT NULL DEFAULT now(),
    content      text        NOT NULL,
    embedding    vector(1536)                    -- must match the embedding model
);

CREATE INDEX ON doc_chunks (tenant_id, language);   -- metadata filters
```

Design rules: keep the **filter columns** (tenant, language, date, document type) as real typed columns rather than inside a JSON blob, store the chunk text next to its vector so retrieval is a single round trip, and record the **model name and dimension** you used so you can detect mismatches later.

5.3 Generating embeddings — in the app or in the database

```
-- azure_ai lets PostgreSQL call Azure OpenAI directly
SELECT azure_ai.set_setting('azure_openai.endpoint', 'https://oai-ai200.openai.azure.com');
SELECT azure_ai.set_setting('azure_openai.subscription_key', '<key>');

UPDATE doc_chunks
SET embedding = azure_openai.create_embeddings('text-embedding-3-small', content)::vector
WHERE embedding IS NULL;
```

That is elegant for backfills. For high-volume ingestion, generate embeddings in the application or a Function so you can batch, retry, and control cost.

5.4 Vector indexes and the operators

pgvector distance operators — the operator you query with **must match** the operator class you indexed with:

Operator	Distance	Index operator class
`<=>`	Cosine	`vector_cosine_ops`
`<->`	L2 / Euclidean	`vector_l2_ops`
`<#>`	Negative inner product	`vector_ip_ops`

```
-- HNSW: better recall and latency, slower to build, more memory
CREATE INDEX ON doc_chunks USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- IVFFlat: faster to build, smaller, needs representative data before building
CREATE INDEX ON doc_chunks USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 200);
```

Query-time recall/latency knobs:

```
SET hnsw.ef_search = 40;      -- higher = better recall, slower
SET ivfflat.probes = 10;      -- higher = better recall, slower
```

	HNSW	IVFFlat
Build time	Slow	Fast
Memory	High	Lower
Recall at speed	Better	Good
Needs data before building	No	**Yes** — build after loading representative rows
Typical choice	Production RAG	Large, frequently rebuilt sets

> **COMMON EXAM TRAP**
> An IVFFlat index built on an **empty or tiny** table produces poor clusters and bad recall forever after. Load the data first, then build. And remember both are **approximate** — recall below 100% is expected, and `ef_search` / `probes` is how you buy it back.

5.5 Similarity search with metadata filters (the RAG query)

```
SELECT id, doc_id, content,
       1 - (embedding <=> $1) AS cosine_similarity
FROM   doc_chunks
WHERE  tenant_id = $2
  AND  language  = 'en'
  AND  updated_at > now() - interval '18 months'
ORDER  BY embedding <=> $1
LIMIT  5;
```

Filtering **before** the vector comparison is cheaper and safer (it is also how you enforce tenant isolation). Where selectivity is very high, a **partial index** per tenant or language avoids scanning the whole vector index:

```
CREATE INDEX ON doc_chunks USING hnsw (embedding vector_cosine_ops)
    WHERE language = 'en';
```

5.6 Compute, memory, and connections

Setting	Guidance for vector workloads
Tier	Burstable for dev only. **Memory Optimized** for HNSW at scale — the index wants to live in RAM
`maintenance_work_mem`	Raise it before building a large HNSW index, or the build crawls
`shared_buffers` / RAM	If the index does not fit in memory, every query hits disk and latency collapses
Storage IOPS	Provisioned IOPS for write-heavy ingestion
Connections	Postgres connections are processes, not threads — they are expensive

Connection optimisation is explicitly in the outline:

- Use the application-side **connection pool** (SQLAlchemy `QueuePool`, `psycopg_pool`) and keep it warm; never open a connection per request.
- Enable built-in **PgBouncer** (transaction pooling, port 6432) when you have many short-lived clients such as serverless functions.
- Prefer **prepared statements** and batch inserts (`COPY` or `execute_values`) for bulk embedding loads.

Reference: compute options and index tuning.

> **KNOW THE DIFFERENCE**
>
> **Cosmos DB vs PostgreSQL for vectors.** Cosmos DB: global distribution, elastic RU scaling, JSON documents, schema-free — best when the application is already document-oriented and needs multi-region writes. PostgreSQL + pgvector: relational joins, rich SQL filtering, mature indexing, lower cost at moderate scale — best when the metadata is relational and the team knows SQL. Redis: sub-millisecond lookups and caching, not durable primary storage.

5.7 Chapter 5 self-check

1. `CREATE EXTENSION vector` fails. What did you skip?
2. Which index type needs representative data loaded before you build it?
3. Recall is too low on an HNSW index. Which single setting do you raise first?
4. 400 serverless function instances exhaust the connection limit. What do you enable?
5. Why keep `tenant_id` as a column instead of inside a JSON document?

> **Answers.** 1. Allow-listing the extension in the `azure.extensions` server parameter. 2. IVFFlat. 3. `hnsw.ef_search` at query time (or rebuild with a larger `m` / `ef_construction`). 4. Built-in PgBouncer with transaction pooling. 5. So it can be indexed and used as a cheap pre-filter before the vector comparison — and so tenant isolation is enforceable.

5.8 Learn more

- Azure Database for PostgreSQL flexible server · pgvector · generative AI overview · Azure OpenAI integration · compute · PgBouncer · index tuning
- Training: Build AI apps with Azure Database for PostgreSQL · Enable semantic search · Build RAG applications

---

Chapter 6 — Azure Managed Redis in AI solutions

> **EXAM FOCUS**
> **Integrate Azure Managed Redis in AI solutions** — implement Azure Managed Redis data operations, including caching, expiration, and invalidation · implement vector indexing to enable similarity search.

6.1 What Redis is for in an AI application

Azure Managed Redis is in-memory, single-digit-millisecond storage. In an AI system it does four jobs:

Job	Pattern
Response cache	Cache an expensive completion keyed by the normalised prompt
Semantic cache	Vector-match a *similar* previous question and reuse its answer
Session / chat history	Short-lived conversation state with a TTL
Rate limiting and locks	Counters with expiry, distributed locks

Tiers: **Memory Optimized** (most RAM per vCPU — vector indexes and large caches), **Balanced**, **Compute Optimized** (throughput-heavy), and **Flash Optimized** (large datasets on NVMe at lower cost per GB).

6.2 Data operations, expiration, and invalidation

```
import redis
r = redis.Redis(host="redis-ai200.<region>.redis.azure.net", port=10000, ssl=True,
                username="default", password=token, decode_responses=True)

# Cache-aside with an expiry
key = f"answer:{hash_prompt(prompt)}"
cached = r.get(key)
if cached is None:
    answer = call_model(prompt)
    r.set(key, answer, ex=900)          # EX = 900 seconds TTL
else:
    answer = cached

r.delete(key)                            # explicit invalidation on source change
r.expire(f"chat:{session_id}", 1800)     # sliding-ish session window
```

Invalidation strategies, and when each is correct:

Strategy	Use when
TTL only	Slightly stale data is acceptable — the default for AI answers
Explicit delete on write	The source of truth changed and staleness is unacceptable
Key versioning (`doc:v7:123`)	You want an instant, atomic "invalidate everything" by bumping the version
Keyspace notifications	Another component must react to expiry or eviction

COMMON EXAM TRAP

A cache with **no expiry** is a memory leak with extra steps. When memory fills, the eviction policy (typically `allkeys-lru`) starts discarding keys you may have assumed were durable. Redis is a cache and a working set — never the system of record.

6.3 Vector indexing and similarity search

Redis supports vector fields in a search index, so it can serve as a low-latency vector store or a semantic cache.

```
FT.CREATE idx:chunks ON HASH PREFIX 1 chunk:
  SCHEMA
    tenant   TAG
    content  TEXT
    embedding VECTOR HNSW 6
      TYPE FLOAT32 DIM 1536 DISTANCE_METRIC COSINE

FT.SEARCH idx:chunks "(@tenant:{acme})=>[KNN 5 @embedding $vec AS score]"
  PARAMS 2 vec <binary-vector> SORTBY score DIALECT 2
```

Choice	Effect
`FLAT`	Exact KNN, brute force. Small sets, guaranteed recall
`HNSW`	Approximate, fast at scale; tune with `M` and `EF_CONSTRUCTION`, and `EF_RUNTIME` per query
`DIM`	Must equal the embedding model's dimensions
`DISTANCE_METRIC`	`COSINE` for OpenAI embeddings
TAG pre-filter	Restricts the candidate set before KNN — the same "filter first" rule as everywhere else

KNOW THE DIFFERENCE

Semantic cache vs vector database. A semantic cache in Redis stores *question → answer* pairs and returns a previous answer when a new question is close enough (a similarity threshold you choose). A vector database stores *document chunks* to ground new answers. An AI app often has both: Redis in front for repeats, Cosmos DB or PostgreSQL behind for knowledge.

6.4 Chapter 6 self-check

1. Which tier suits a large HNSW vector index?
2. A cached answer must disappear the moment its source document is edited. Which strategy?
3. What must `DIM` equal in `FT.CREATE`?
4. Why is `FLAT` sometimes the right index type?
5. What breaks when keys are written without a TTL?

> **Answers.** 1. Memory Optimized. 2. Explicit delete on write (or key versioning), not TTL alone. 3. The embedding model's dimension count, for example 1536. 4. It gives exact recall on small collections with no tuning. 5. Memory fills and the eviction policy silently drops keys.

6.5 Learn more

- Azure Managed Redis · Azure Cache for Redis overview · vector similarity tutorial
- Training: Enhance AI solutions with Azure Managed Redis · data operations · vector storage · event messaging

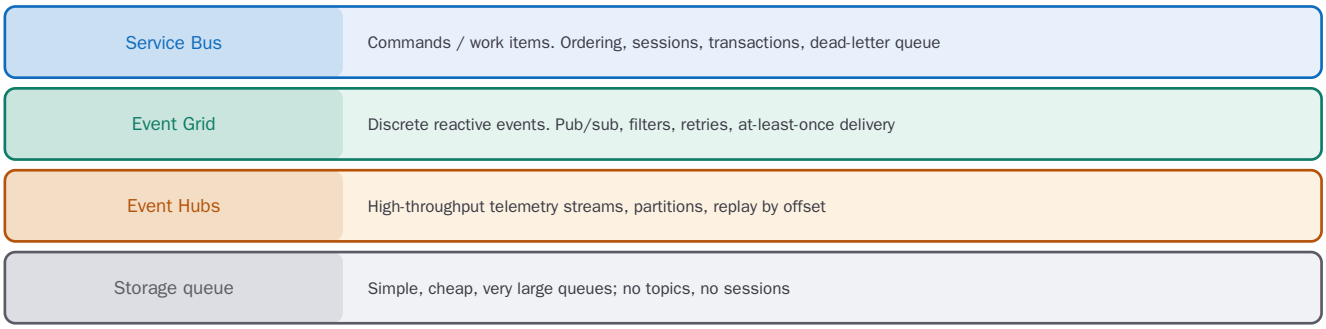
---

Chapter 7 — Event- and message-based AI solutions

EXAM FOCUS

Develop event- and message-based AI solutions — queue and process back-end operations by using Azure Service Bus, including dead-letter queue handling, messages, topics, and subscriptions · implement event-driven workflows by using Azure Event Grid, including filters, custom events, and retries.

7.1 Choosing the right service

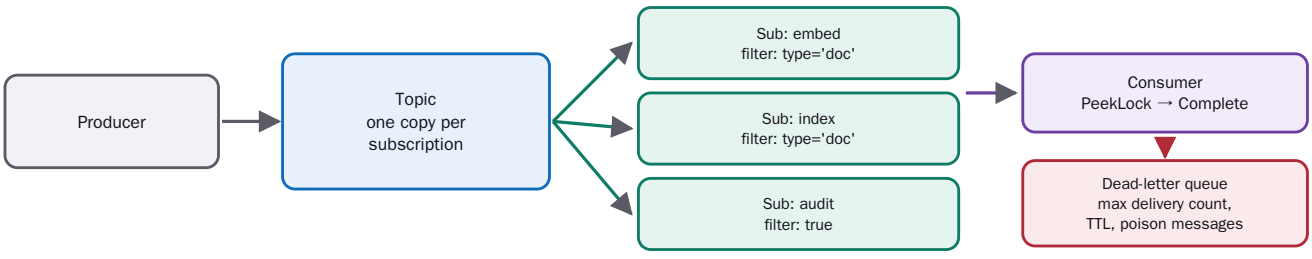


Four messaging services. The give-away is whether the payload is a command that must be processed or a notification that something happened.

KNOW THE DIFFERENCE

Service Bus vs Event Grid in one line each. Service Bus carries **messages**: "embed this document" — the sender cares that it is processed, exactly once, in order, with retries and a dead-letter queue. Event Grid carries **events**: "a blob was created" — the publisher does not care who reacts, delivery is at-least-once, and handlers must be idempotent.

7.2 Azure Service Bus



A topic fans one message out to every matching subscription; poison messages end up in the dead-letter queue.

Concept	What to know
Queue	Point-to-point. One competing-consumer group drains it
Topic + subscriptions	Publish-subscribe. **Each subscription gets its own copy** and has its own filter and its own DLQ
Filters	*SQL filter* (expressions over properties), *correlation filter* (cheapest, matches on system/user properties), *boolean* true/false
Sessions	FIFO ordering and state for a group of related messages (for example one chat conversation)
PeekLock	Message is locked, not deleted; you then `complete`, `abandon`, `defer`, or `dead-letter` it
ReceiveAndDelete	Deleted on receive — faster, but a crash loses the message
Duplicate detection	Ignores repeats of the same `MessageId` within a time window
Scheduled messages	Enqueue now, deliver later

```
from azure.servicebus import ServiceBusClient, ServiceBusMessage
from azure.identity import DefaultAzureCredential

client = ServiceBusClient("sb-ai200.servicebus.windows.net", DefaultAzureCredential())

with client.get_queue_sender("embed-jobs") as sender:
    sender.send_messages(ServiceBusMessage(
        body=json.dumps({"docId": doc_id}),
        message_id=doc_id,                      # enables duplicate detection
        application_properties={"type": "pdf", "tenant": "acme"}))

with client.get_queue_receiver("embed-jobs", max_wait_time=30) as receiver:
    for msg in receiver:
        try:
            handle(json.loads(str(msg)))
            receiver.complete_message(msg)       # done, remove it
        except TransientError:
            receiver.abandon_message(msg)        # retry, delivery count +1
        except PoisonError as e:
            receiver.dead_letter_message(msg, reason="unparseable",
error_description=str(e))
```

Dead-lettering is explicitly in the outline. A message lands in the DLQ when:

- the **max delivery count** is exceeded (repeated abandon or lock expiry),
- the message **TTL** expires,
- a **subscription filter evaluation** throws,
- the application calls `dead_letter_message` explicitly.

The DLQ is a real sub-queue at `<queue>/$deadletterqueue` — you read it, fix the cause, and resubmit. Reference: dead-letter queues.

> **COMMON EXAM TRAP**
>
> If processing takes longer than the **lock duration** (default 30 seconds, max 5 minutes), the lock expires, the message reappears, the delivery count climbs, and it eventually dead-letters — while your handler is still running and may complete the work several times. Fix it with lock renewal, a longer lock duration, or by making the handler idempotent. Embedding a large document is exactly this scenario.

7.3 Azure Event Grid

Event Grid is a fully managed pub/sub router for discrete events, from Azure sources (Blob Storage, Event Hubs, resource groups) or your own **custom topics**.

```
from azure.eventgrid import EventGridPublisherClient, EventGridEvent

client = EventGridPublisherClient(topic_endpoint, DefaultAzureCredential())
client.send([EventGridEvent(
    subject="documents/acme/policy-2026.pdf",
    event_type="Contoso.Docs.Uploaded",
    data={"docId": doc_id, "pages": 42, "tenant": "acme"},
    data_version="1.0")])
```

Filtering happens in the subscription, so handlers only receive what they care about:

Filter	Example
Event type	`Contoso.Docs.Uploaded`
Subject begins/ends with	begins `documents/acme/`, ends `.pdf`
Advanced filter	`data.pages` `NumberGreaterThan` `10`; `data.tenant` `StringIn` `[acme, fabrikam]`

Retries and reliability:

- Delivery is **at-least-once** with **exponential back-off**; the default retry policy runs for up to **24 hours / 30 attempts**, whichever you hit first, and both are configurable.
- Undeliverable events go to a **dead-letter destination** (a storage container) — you must configure it, or the events are dropped.
- Webhook subscribers must complete a **validation handshake** unless they are an Azure service or use an approved validation method.

Reference: event filtering, delivery and retry, event schema, and custom events quickstart.

> **COMMON EXAM TRAP**
>
> At-least-once means **duplicates are normal**, and events can arrive **out of order**. If a question says "the document must be embedded exactly once", the answer is an idempotency key in your handler (or Service Bus duplicate detection), not "configure Event Grid to deliver once" — that option does not exist.

7.4 Putting them together

A common exam architecture: Blob upload → **Event Grid** system topic → **Service Bus queue** → Container App scaled by **KEDA** → embeddings written to Cosmos DB.

Why both? Event Grid reacts instantly and cheaply to the storage event; Service Bus then gives the worker retry, ordering, back-pressure and a dead-letter queue. Event Grid alone has no back-pressure; Service Bus alone cannot observe blob creation.

7.5 Chapter 7 self-check

1. Three teams must each receive every "document uploaded" message and apply their own filter. Queue or topic?
2. A handler takes 4 minutes; messages keep reappearing and dead-lettering. Why?
3. Events that cannot be delivered must be retained for investigation. What do you configure?
4. Which filter type matches on `data.pages > 10`?
5. Which service gives back-pressure for a slow worker?

> **Answers.** 1. A topic with three subscriptions, each with its own filter. 2. The lock duration is shorter than the processing time — renew the lock, extend the duration, or make the work idempotent and shorter. 3. A dead-letter destination (storage container) on the Event Grid subscription. 4. An advanced filter (`NumberGreaterThan`). 5. Service Bus — the queue absorbs the backlog and the consumer pulls at its own rate.

7.6 Learn more

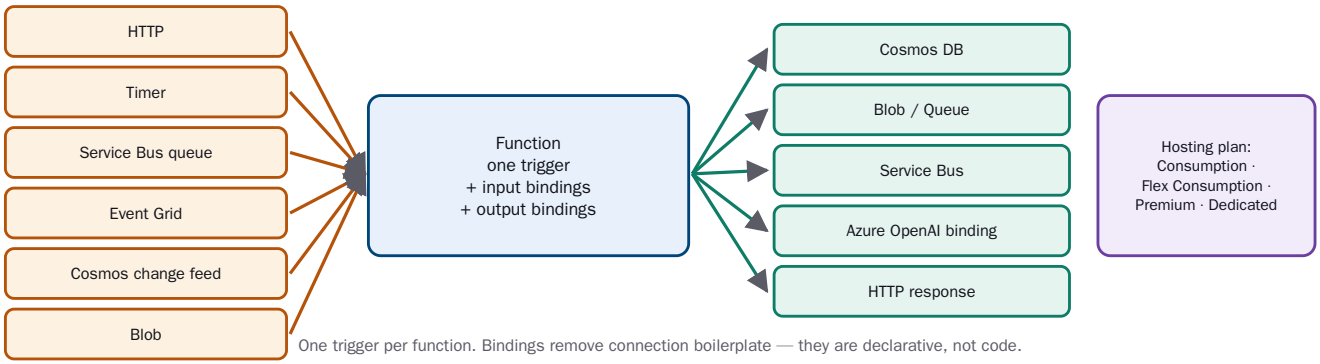
- Service Bus overview · queues, topics, subscriptions · dead-letter queues · topic filters · message sessions
- Event Grid overview · concepts · filtering · delivery and retry · event schema · MQTT
- Training: Queue and process AI operations with Azure Service Bus · Develop event-driven AI workflows with Event Grid

---

Chapter 8 — Azure Functions

> **EXAM FOCUS**
>
> **Develop and implement Azure Functions** — build serverless APIs, including implementing triggers and bindings · configure and deploy function apps.

8.1 The trigger and binding model



One trigger starts the function; bindings connect it to data without connection-handling code.

Every function has **exactly one trigger** and any number of **input** and **output** bindings. Bindings are declarative: the platform handles the connection, retries and serialisation.

```
import azure.functions as func
app = func.FunctionApp(http_auth_level=func.AuthLevel.FUNCTION)

# HTTP-triggered API with a Cosmos DB output binding
@app.route(route="chunks", methods=["POST"])
@app.cosmos_db_output(arg_name="out", connection="COSMOS_CONN",
                      database_name="knowledge", container_name="chunks")
def add_chunk(req: func.HttpRequest, out: func.Out[func.Document]) -> func.HttpResponse:
    body = req.get_json()
    out.set(func.Document.from_dict(body))
    return func.HttpResponse(status_code=201)

# Service Bus trigger with automatic retry and dead-lettering by the platform
@app.service_bus_queue_trigger(arg_name="msg", queue_name="embed-jobs",
                               connection="SB_CONN")
def embed(msg: func.ServiceBusMessage):
    doc = json.loads(msg.get_body().decode())
    store_vector(doc["docId"], embed_text(doc["text"]))
```

Triggers you should recognise: HTTP, Timer (CRON), Blob, Queue Storage, Service Bus (queue and topic), Event Grid, Event Hubs, **Cosmos DB change feed**, and Durable orchestration triggers.

> **TIP**
> Azure Functions has an **Azure OpenAI extension** with bindings for text completion, embeddings, assistants, and semantic search — a genuinely low-code way to build the embed-on-upload step, and exactly the sort of "AI back-end" detail this exam likes.

8.2 Hosting plans

Plan	Scale to zero	Cold start	VNet	Use it when
Consumption	Yes	Yes	Limited	Spiky, low-volume, cost-sensitive
Flex Consumption	Yes	Reduced (always-ready instances)	Yes	Modern default: serverless scale plus VNet and per-instance concurrency control
Premium	No (pre-warmed)	No	Yes	Predictable low latency, long runs, VNet
Dedicated (App Service plan)	No	No	Yes	Reuse an existing plan, or need its features
Container Apps hosting	Yes	Yes	Yes	Functions alongside other containers in one environment

Reference: hosting and scale.

8.3 Configuration and deployment

- **Application settings** are the environment variables. `AzureWebJobsStorage` (the required storage account for state, timers and leases) and `FUNCTIONS_WORKER_RUNTIME` are mandatory.
- Use **Key Vault references** for anything secret, resolved through the app's managed identity.
- Prefer **identity-based connections** (`SB_CONN__fullyQualifiedNamespace`) over connection strings where the binding supports them.
- Deploy with **zip deploy / run-from-package** (`WEBSITE_RUN_FROM_PACKAGE=1`) for an atomic, read-only deployment — the recommended production pattern. See deployment technologies.
- Control cost and downstream pressure with `maxConcurrentCalls` / `batchSize` in `host.json` and a **maximum scale-out** limit.

> **COMMON EXAM TRAP**
> A Functions app **requires a storage account**. Timer triggers, singleton locks, Event Hubs checkpoints and Durable state all live there. Locking that storage account behind a firewall without the correct private endpoint and exemptions is a classic "the function stopped triggering and the logs are empty" scenario.

8.4 Durable Functions for long AI workflows

Durable Functions add stateful orchestration — essential when one request triggers many model calls.

```
@app.orchestration_trigger(context_name="context")
def ingest_orchestrator(context):
    doc = context.get_input()
    chunks = yield context.call_activity("split_document", doc)
    # fan out: embed every chunk in parallel, then fan in
    tasks = [context.call_activity("embed_chunk", c) for c in chunks]
    vectors = yield context.task_all(tasks)
    return (yield context.call_activity("write_index", vectors))
```

Patterns to recognise: **function chaining**, **fan-out/fan-in** (the one above — parallel embedding), **async HTTP API** (long job + status endpoint), **monitor**, and **human interaction** with a timeout.

> **WATCH OUT**
> Orchestrator code must be **deterministic**: no `datetime.now()`, no random values, no direct I/O, no non-deterministic iteration order. It is replayed from the event history on every await, so non-deterministic code produces corrupt state. Put all side effects in activity functions.

8.5 Chapter 8 self-check

1. How many triggers can a function have?
2. A function must run in a VNet, scale to zero, and avoid long cold starts. Which plan?
3. A timer trigger stopped firing and there are no logs. What do you check first?
4. 200 chunks must be embedded in parallel and combined. Which Durable pattern?
5. Why must an orchestrator never call `datetime.now()`?

> **Answers.** 1. Exactly one. 2. Flex Consumption. 3. The storage account (`AzureWebJobsStorage`) — connectivity, firewall rules, or a deleted account. 4. Fan-out/fan-in with `task_all`. 5. Orchestrators are replayed; non-deterministic values break replay. Use `context.current_utc_datetime`.

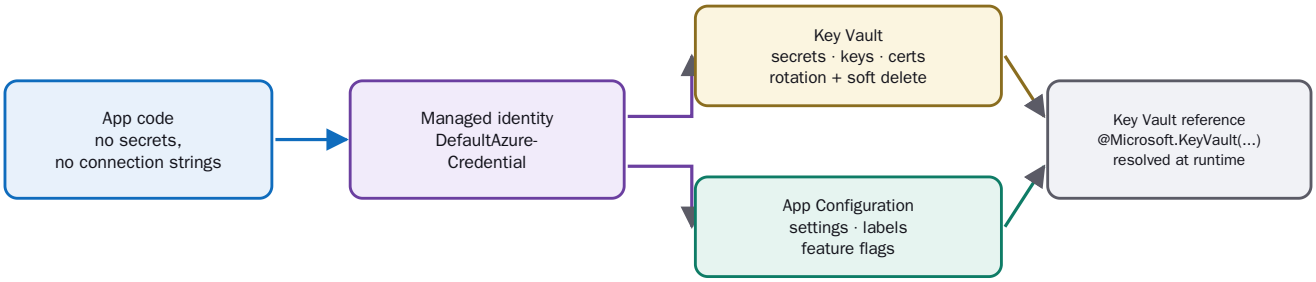
8.6 Learn more

- Azure Functions overview · triggers and bindings · hosting and scale · Python developer guide · deployment technologies · Azure OpenAI bindings · local development
- Durable Functions
- Training: Build serverless AI backends with Azure Functions · Orchestrate durable AI workflows · Integrate backend services for AI solutions

---

Chapter 9 — Secrets and configuration

> **EXAM FOCUS**
>
> **Implement secure Azure solutions** — secure secrets by using Azure Key Vault, including rotation and retrieval · store and retrieve app configuration information by using Azure App Configuration.



App Configuration stores the pointer; Key Vault stores the secret. Neither is in source control.

Managed identity fetches settings from App Configuration and secrets from Key Vault. Neither is in the code or the repo.

9.1 Azure Key Vault

Key Vault stores three object types: **secrets** (strings), **keys** (cryptographic operations, never exported), and **certificates**.

Topic	What to know
Authorisation	**Azure RBAC** is the recommended model (`Key Vault Secrets User` to read, `Key Vault Secrets Officer` to write). Legacy access policies still exist; RBAC is the modern answer
Soft delete	Always on. Deleted objects are recoverable for the retention period
Purge protection	Optional but recommended; prevents permanent deletion during the retention window
Versions	Every update creates a version. Reference without a version to always get the current one
Networking	Firewall, service endpoints, or private endpoint with "trusted Microsoft services" exemptions

```
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential

client = SecretClient("https://kv-ai200.vault.azure.net/", DefaultAzureCredential())
api_key = client.get_secret("openai-key").value        # cache this — do not call per
request
```

Rotation is explicitly in the outline. The supported pattern:

1. Set a **rotation policy** with an expiry and a "near expiry" window on the secret.
2. Key Vault emits a `SecretNearExpiry` **event to Event Grid**.

3. An **Azure Function** subscribed to that event generates a new credential in the source system and writes the new secret version.
4. Applications that read the secret **without a version** pick up the new value — provided they refresh their cache.

See secret rotation tutorial and the Key Vault developer's guide.

> **COMMON EXAM TRAP**
> Fetching a secret from Key Vault **on every request** is the classic failure: it adds latency and you will hit throttling (HTTP 429) under load. Cache the secret in memory with a refresh interval, or use a Key Vault reference so the platform caches it for you.

9.2 Azure App Configuration

App Configuration is for settings that are **not** secrets: endpoints, model names, timeouts, chunk sizes, and feature flags.

Concept	Detail
Key-value	Hierarchical keys such as `Chat:Model`, `Chat:TopK`
Label	The same key with a different value per environment — `Dev`, `Test`, `Prod`
Key Vault reference	A key whose value is a pointer to a Key Vault secret; App Configuration never stores the secret itself
Feature flag	Managed on/off switches with filters (percentage, time window, targeting) — see feature management
Sentinel key	One key you bump after a batch of changes; clients watch only that key and then reload everything at once
Snapshot	An immutable point-in-time set of key-values you can pin a release to

```
from azure.appconfiguration.provider import load, WatchKey

config = load(
    endpoint="https://appcs-ai200.azconfig.io",
    credential=DefaultAzureCredential(),
    selects=[SettingSelector(key_filter="Chat:*", label_filter="Prod")],
    refresh_on=[WatchKey("Sentinel")],          # reload when the sentinel changes
    refresh_interval=30,
    key_vault_options=...,                       # resolves Key Vault references
)
top_k = int(config["Chat:TopK"])
```

> **KNOW THE DIFFERENCE**
>
> **Key Vault vs App Configuration.** If leaking it would be a security incident, it is a **secret** → Key Vault. If it is an operational value you want to change without redeploying, it is **configuration** → App Configuration. The two are designed to be used together: App Configuration holds the reference, Key Vault holds the value.

9.3 Chapter 9 self-check

1. Which RBAC role lets an app read secrets but not write them?
2. What triggers the rotation workflow?
3. How do you change `Chat:TopK` in production without redeploying?
4. How do you make thirty settings take effect together, atomically?
5. Where should an Azure OpenAI API key live — and what is better than storing it at all?

> **Answers.** 1. `Key Vault Secrets User`. 2. The `SecretNearExpiry` Event Grid event raised by the rotation policy. 3. Update the key in App Configuration with the `Prod` label; the provider refreshes it. 4. Update a sentinel key (or pin to a snapshot) so clients reload as one. 5. Key Vault — but better still, use **managed identity with RBAC on the Azure OpenAI resource** so there is no key.

9.4 Learn more

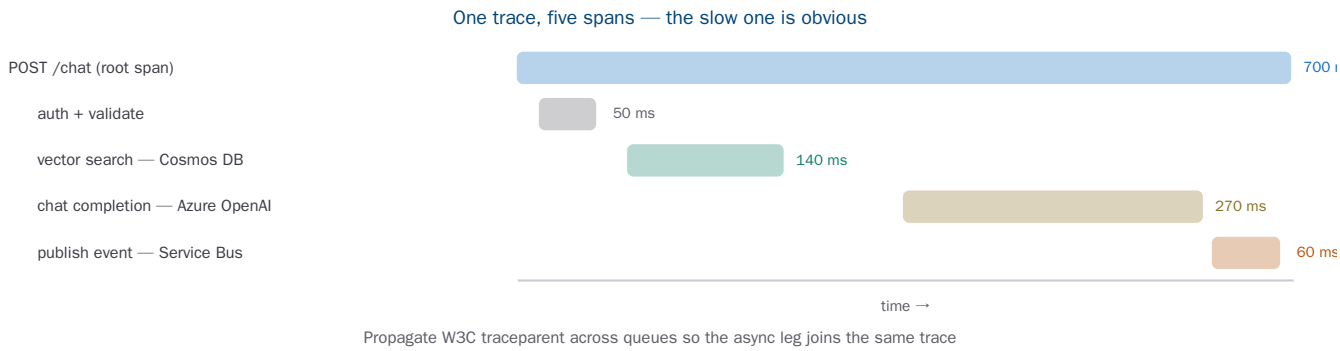
- Key Vault overview · about secrets · RBAC guide · rotation tutorial · developer's guide
- App Configuration overview · key-values · best practices · feature management
- Training: Manage application secrets and configuration for AI solutions · Key Vault · App Configuration

---

Chapter 10 — Monitoring and troubleshooting

> **EXAM FOCUS**
>
> **Monitor and troubleshoot Azure solutions** — trace distributed systems by using OpenTelemetry SDKs · write KQL queries to analyze logs and metrics.

10.1 OpenTelemetry and distributed tracing



One request, five spans. Distributed tracing is how you prove the model call — not your code — is the slow part.

Azure Monitor's supported path is the **Azure Monitor OpenTelemetry Distro**: standard OpenTelemetry APIs, exported to Application Insights.

```
from azure.monitor.opentelemetry import configure_azure_monitor
from opentelemetry import trace

configure_azure_monitor(connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"
])
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("retrieve_context") as span:
    span.set_attribute("tenant.id", tenant)
    span.set_attribute("retrieval.top_k", top_k)
    chunks = vector_search(query_vector, top_k)
    span.set_attribute("retrieval.hit_count", len(chunks))
```

The three signals, and what each answers:

Signal	Answers
Traces (spans)	"Where did the 4 seconds go?" — a parent span with child spans per dependency
Metrics	"How is it trending?" — request rate, duration percentiles, token counts
Logs	"What exactly happened in this one failure?"

Concepts the exam expects:

- **Context propagation** uses the W3C `traceparent` header. Propagate it through **queues and events** too (as a message property), or the async half of the workflow becomes an orphan trace.
- **Auto-instrumentation** covers common libraries (HTTP clients, Azure SDKs, database drivers); add **manual spans** around your own logic — chunking, retrieval, prompt assembly.
- **Sampling** controls cost; set it deliberately rather than discovering the bill later.
- Instrument once per process, at startup, not per request.

Reference: OpenTelemetry overview, distributed tracing, and Application Insights overview.

10.2 KQL for logs and metrics

Application Insights and Log Analytics are queried with KQL. The tables you must recognise:

Table	Contents
`requests`	Incoming calls: name, duration, resultCode, success
`dependencies`	Outgoing calls: HTTP, Cosmos DB, Service Bus, Azure OpenAI
`exceptions`	Thrown exceptions with stack traces
`traces`	Your log statements
`customMetrics` / `customEvents`	Values and events you emit yourself
`ContainerAppConsoleLogs_CL`, `ContainerLogV2`	Container stdout/stderr

```
// 1. Slowest operations in the last hour, with p95
requests
| where timestamp > ago(1h)
| summarize count(), avg(duration), p95 = percentile(duration, 95) by name
| order by p95 desc

// 2. Which dependency is causing the slow /chat endpoint?
dependencies
| where timestamp > ago(1h) and operation_Name == "POST /chat"
| summarize calls = count(), p95 = percentile(duration, 95) by target, type
| order by p95 desc

// 3. Failure rate over time, five-minute buckets
requests
| where timestamp > ago(6h)
| summarize total = count(), failed = countif(success == false) by bin(timestamp, 5m)
| extend failureRate = 100.0 * failed / total
| render timechart

// 4. Follow one failed request end to end
let op = "a1b2c3d4e5f6";
union requests, dependencies, exceptions, traces
| where operation_Id == op
| project timestamp, itemType, name, target, resultCode, duration, message
| order by timestamp asc

// 5. Throttling (429) from any dependency
dependencies
| where timestamp > ago(24h) and resultCode == "429"
| summarize throttled = count() by target, bin(timestamp, 1h)
| render columnchart
```

Operators to have at your fingertips: `where`, `project`, `extend`, `summarize … by bin()`, `top … by`, `order by`, `join kind=inner`, `union`, `let`, `render`, `percentile()`, `countif()`, `ago()`, `arg_max()`.

COMMON EXAM TRAP

`operation_Id` correlates **all telemetry for one logical operation**; `operation_ParentId` gives the parent-child relationship inside it. A question that says "trace a single user request across the API, the queue, and the worker" is asking for `operation_Id`, not a timestamp range.

10.3 A troubleshooting playbook

Symptom	Where to look	Common cause
Endpoint slow, CPU low	`dependencies` p95 by target	The model call or a cross-partition Cosmos query
Intermittent 500s	`exceptions` grouped by type	Transient throttling not retried, or a null from a missing config value
429 from Cosmos DB	`dependencies` where resultCode 429	Hot partition or under-provisioned RU/s
401/403 to Key Vault or Cosmos	`exceptions`, resource logs	Missing **role assignment** for the managed identity
Worker idle while the queue grows	Service Bus metrics, KEDA scale events	Scale rule not configured, or `maxReplicas` too low
Trace stops at the queue	Missing `traceparent` propagation	Not copying the context into the message properties

10.4 Chapter 10 self-check

1. Which package configures OpenTelemetry export to Application Insights in Python?
2. Which field links the API span to the worker span for the same request?
3. Which table holds outbound calls to Azure OpenAI?
4. Write the shape of a query returning p95 duration by dependency target.
5. Traces stop at the queue boundary. What is missing?

> **Answers.** 1. `azure-monitor-opentelemetry` (`configure_azure_monitor()`). 2. `operation_Id` (with `operation_ParentId` for hierarchy). 3. `dependencies`. 4. `dependencies | summarize percentile(duration, 95) by target`. 5. W3C `traceparent` propagation into the message properties and back out in the consumer.

10.5 Learn more

- Enable OpenTelemetry · OpenTelemetry overview · distributed tracing · Application Insights
- Log queries overview · get started with queries · Log Analytics tutorial
- Training: Observe and troubleshoot apps on Azure · Instrument an app with OpenTelemetry · Analyze app telemetry with logs and metrics

---

Chapter 11 — The hands-on lab track

AI-200 is a developer exam: it tests things you only truly know after typing them. Work through this list in order — every item is a free guided Microsoft Learn exercise.

#	Lab	Skill area it proves
1	Store and manage containers in Azure Container Registry	ACR, tags, tasks
2	Deploy containers to Azure App Service	App Service containers, settings
3	Run sidecar-enabled AI applications on App Service	Sidecar pattern
4	Deploy containers to Azure Container Apps	Environments, ingress
5	Manage containers in Azure Container Apps	Revisions, traffic splitting
6	Scale containers in Azure Container Apps	KEDA scale rules, scale to zero
7	Run AI-generated code in dynamic sessions	Sandboxed code execution
8	Deploy an Azure Kubernetes Service cluster	AKS provisioning
9	Deploy applications to AKS	Manifests, kubectl
10	Configure scaling in AKS	HPA, cluster autoscaler, KEDA
11	Build queries for Azure Cosmos DB for NoSQL	SQL API queries
12	Implement vector search on Cosmos DB	Vector policy, `VectorDistance()`
13	Optimize query performance for Cosmos DB	RUs, indexing policy
14	Handle events with Functions and the change feed	Change feed processor
15	Get started with generative AI in PostgreSQL	azure_ai extension
16	Enable semantic search in PostgreSQL	pgvector, HNSW
17	Build RAG applications with PostgreSQL	End-to-end RAG
18	Implement data operations in Azure Managed Redis	Caching, TTL, invalidation
19	Implement vector storage in Azure Managed Redis	`FT.CREATE`, KNN
20	Queue and process AI operations with Service Bus	Queues, topics, DLQ
21	Develop event-driven AI workflows with Event Grid	Filters, retries
22	Build serverless AI backends with Azure Functions	Triggers, bindings
23	Orchestrate durable AI workflows	Fan-out/fan-in
24	Manage application secrets with Key Vault	Secrets, rotation
25	Manage application settings with App Configuration	Labels, feature flags
26	Instrument an app with OpenTelemetry	Spans, propagation
27	Analyze app telemetry with logs and metrics	KQL

HANDS-ON LAB

Build this once from scratch, without instructions. Upload a PDF to Blob Storage → Event Grid raises an event → a Function puts a job on a Service Bus queue → a Container App with a KEDA queue scale rule (min 0) chunks and embeds it → vectors land in Cosmos DB with a vector index → a chat API does a filtered vector search and calls Azure OpenAI → Redis caches repeated answers → all secrets in Key Vault via managed identity → one OpenTelemetry trace spans the whole thing. If you can build that, you can pass this exam.

Chapter 12 — Practice questions

1. A base image security patch must automatically rebuild every dependent image, with no pipeline changes. What do you configure?
2. Staging and production slots must point at different databases, and a swap must not exchange them. What do you set?
3. A worker must cost nothing overnight and scale on Service Bus queue depth. Which configuration?
4. Which Container Apps scale rule types can scale to zero?
5. A pod is `CrashLoopBackOff`. Which command shows why?
6. Which Cosmos DB read costs about 1 RU?
7. Write RUs are high on a container holding 1536-float embeddings. First fix?
8. Deleted Cosmos DB documents still appear in search results built from the change feed. Why?
9. An `ORDER BY category, updatedAt DESC` query fails in Cosmos DB. What is missing?
10. Which Cosmos DB consistency level is the default, and what does it guarantee?
11. `CREATE EXTENSION vector` fails on PostgreSQL flexible server. What was skipped?
12. Which pgvector index needs representative data loaded before it is built?
13. Which pgvector operator pairs with `vector_cosine_ops`?
14. 300 function instances exhaust PostgreSQL connections. What do you enable?
15. A cached LLM answer must be invalidated the instant its source document changes. Which strategy?
16. Three teams must each receive every upload message and filter it differently. Queue or topic?
17. Messages reappear and eventually dead-letter while the handler is still running. Why?
18. Event Grid events that cannot be delivered must be kept for investigation. What do you configure?
19. How many triggers may one Azure Function have?
20. Which Functions hosting plan gives VNet integration, scale to zero, and reduced cold starts?
21. Why must a Durable orchestrator avoid `datetime.now()`?
22. An app fetches a Key Vault secret on every request and starts getting 429s. Fix?
23. Which Event Grid event drives the secret rotation workflow?
24. How do you apply thirty configuration changes atomically?
25. Which KQL field correlates the API span with the worker span for one request?

▸ Answers and explanations

1. **An ACR task with a base image update trigger.** It fires on the registry event, not on a schedule.
2. **Mark the setting as a deployment slot setting**, so it stays with the slot during a swap.
3. **A KEDA `azure-servicebus` scale rule with `minReplicas: 0`** and a sensible `messageCount` and `maxReplicas`.
4. **HTTP, TCP, and custom (KEDA) rules.** CPU and memory rules cannot scale to zero.
5. **`kubectl logs <pod> --previous`** (plus `kubectl describe pod` for events).

6. **A point read** — `read_item` with both the id and the partition key.
7. **Exclude the embedding path** (and other large, never-filtered paths) from the indexing policy.
8. **The default change feed mode does not emit deletes.** Use soft deletes with TTL, or all-versions-and-deletes mode.
9. **A composite index** on `category` ascending and `updatedAt` descending.
10. **Session** — read-your-own-writes within a client session, without the cost of global strong consistency.
11. **Allow-listing `vector` in the `azure.extensions` server parameter** before creating it in the database.
12. **IVFFlat** — it clusters the existing data, so an empty build gives poor recall permanently.
13. `<=>` (cosine distance).
14. **PgBouncer** in transaction pooling mode, plus an application-side pool.
15. **Explicit delete on write** (or key versioning). TTL alone cannot be instant.
16. **A topic with three subscriptions**, each with its own filter and its own dead-letter queue.
17. **The lock duration is shorter than the processing time.** Renew the lock, raise the duration, or shorten/idempotise the work.
18. **A dead-letter destination** (a storage container) on the Event Grid subscription.
19. **Exactly one.**
20. **Flex Consumption.**
21. **Orchestrators are replayed** from event history; non-deterministic values corrupt the replay. Use `context.current_utc_datetime`.
22. **Cache the secret in memory** with a refresh interval, or use a Key Vault reference so the platform caches it.
23. `SecretNearExpiry`, emitted because a rotation policy is set on the secret.
24. **Update a sentinel key** that clients watch (or pin the release to a snapshot), so all thirty land together.
25. `operation_Id` (with `operation_ParentId` giving the hierarchy).

Chapter 13 — Exam-day checklist and quick reference

13.1 The week before

- Re-read the official AI-200 study guide and check whether a practice assessment has been published on the exam page.
- Build the end-to-end solution from the Chapter 11 lab callout from memory, in one sitting.
- Practise reading **error output**: a `kubectl describe` event list, a Service Bus dead-letter reason, an Application Insights failure blade.
- Write five KQL queries without looking anything up.
- Confirm your Learn profile username is correct so the credential indexes properly.

13.2 The 60-second memory dump

Topic	The one line
Identity	Managed identity + RBAC role assignment; `DefaultAzureCredential`
ACR	Premium = geo-replication + private link; admin user stays off
ACR Tasks	Base image update trigger = automatic rebuild on patch
App Service	App settings = env vars; Key Vault references; **slot settings do not swap**
Container Apps	Environment = boundary; revision = version; multiple-revision mode for traffic split
KEDA	Scales on backlog; only HTTP/TCP/custom rules scale to zero
AKS	HPA = pods · cluster autoscaler = nodes · KEDA = event-driven pods
Container errors	ImagePullBackOff = auth/tag · CrashLoop = config · OOMKilled = limits · Pending = capacity
Cosmos RU	Point read ≈ 1 RU; cross-partition queries are the expensive ones
Cosmos indexing	Everything indexed by default — exclude embeddings and big text
Cosmos vectors	Policy set at container creation; dimensions must match the model; diskANN for scale
Change feed	Latest-version mode, no deletes, lease container, at-least-once
Consistency	Session is the default and the usual right answer
pgvector	Allow-list then `CREATE EXTENSION`; `<=>` cosine; HNSW vs IVFFlat
pgvector recall	`hnsw.ef_search` / `ivfflat.probes` raise recall at the cost of latency
Postgres connections	PgBouncer + application pool
Redis	Cache, semantic cache, session state; TTL always; `FT.CREATE … VECTOR HNSW`
Service Bus	Messages/commands; sessions = FIFO; DLQ on max delivery count or TTL
Event Grid	Events/notifications; at-least-once; advanced filters; dead-letter to storage
Functions	One trigger per function; Flex Consumption is the modern default; storage account required
Durable	Fan-out/fan-in for parallel embedding; orchestrators must be deterministic
Key Vault	RBAC roles; soft delete always on; rotate via `SecretNearExpiry` + Function; cache reads
App Configuration	Labels per environment; sentinel key for atomic reload; feature flags
OpenTelemetry	`configure_azure_monitor()`; propagate `traceparent` through queues
KQL	`requests` · `dependencies` · `exceptions` · `traces`; correlate on `operation_Id`

13.3 Reading exam questions

- Read the **last sentence first** — it states the actual requirement.
- **"Minimise administrative effort"** favours the managed option: Container Apps over AKS, bindings over SDK code, Key Vault references over custom fetch logic.
- **"Minimise cost when idle"** means scale to zero: Consumption/Flex Functions, or Container Apps with `minReplicas: 0`.
- **"Must not store credentials"** means managed identity, every time.
- **"Exactly once"** is a handler-side idempotency requirement, never a delivery guarantee you can switch on.
- **"Regardless of load"** and **"predictable latency"** point away from Consumption plans and scale-to-zero.
- Flag and move on. Every question is worth the same.

13.4 Glossary

Term	Meaning
ACR	Azure Container Registry — private registry for container images
ACR Task	Cloud-side image build, optionally triggered by commit, base image update, or schedule
Revision	An immutable snapshot of a Container Apps version; traffic can be split across revisions
Environment (Container Apps)	The network and logging boundary that apps share
KEDA	Kubernetes Event-driven Autoscaling — scales on external metrics such as queue depth
HPA	Horizontal Pod Autoscaler — scales pods on CPU/memory or custom metrics
Cluster autoscaler	Adds and removes AKS nodes when pods cannot be scheduled
Workload identity	Federates a Kubernetes service account to a managed identity
RU	Request Unit — Cosmos DB's normalised cost currency for reads, writes and queries
Logical partition	All items sharing a partition key value; hard limit of 20 GB
Composite index	Cosmos DB index required for multi-property `ORDER BY`
Change feed	Ordered record of inserts and updates in a Cosmos DB container
Lease container	Stores the change feed processor's position and distributes partitions
pgvector	PostgreSQL extension adding the `vector` type and distance operators
HNSW / IVFFlat	Approximate nearest-neighbour index types for vector search
ef_search / probes	Query-time recall knobs for HNSW and IVFFlat
PgBouncer	Connection pooler built into PostgreSQL flexible server
Semantic cache	Returning a previous answer when a new question is semantically close enough
PeekLock	Service Bus receive mode that locks rather than deletes the message
Dead-letter queue	Sub-queue holding messages that could not be processed
Correlation filter	The cheapest Service Bus subscription filter, matching on properties
At-least-once	Delivery guarantee that permits duplicates — handlers must be idempotent
Trigger / binding	What starts a function / declarative connection to data
Durable Functions	Stateful orchestration over Functions; replay-based, must be deterministic

Term	Meaning
Key Vault reference	An app setting whose value is resolved from Key Vault at runtime
Sentinel key	One App Configuration key that clients watch to reload a batch of changes
Span / trace	A timed unit of work / the tree of spans for one logical operation
traceparent	The W3C header that carries trace context between services
operation_Id	The Application Insights field that correlates all telemetry for one operation

13.5 Where to go next

- **Renew** every 12 months with the free renewal assessment.
- **Pair it with AI-300** (Machine Learning Operations Engineer Associate) to cover the operations side: training, evaluation, deployment and monitoring of the models your back end calls. The AI-300 guide in this series follows the same structure.
- Keep current with the Azure updates feed and Azure Architecture Center.
- Ask questions on Microsoft Q&A and Azure community support.

> **KEY TAKEAWAY**
> For every scenario, be able to answer in one sentence: *which host, which data store, which messaging service, how it authenticates, and how I would prove where the latency went.* That is what 700 points looks like.

Appendix A — One-page decision tables

A.1 Which compute host?

Signal in the question	Answer
"Single container, least operational effort, slots"	App Service
"Scale to zero", "queue-driven", "microservices", "minimise ops"	Container Apps
"Kubernetes manifests", "operators", "specific GPU node pools", "service mesh"	AKS
"Run untrusted model-generated code"	Container Apps dynamic sessions
"Event-driven, per-execution billing, small units of work"	Azure Functions

A.2 Which data store?

Signal	Answer
"JSON documents", "global distribution", "elastic throughput"	Cosmos DB for NoSQL
"Relational filters and joins alongside vectors"	PostgreSQL + pgvector
"Sub-millisecond", "cache", "session state", "semantic cache"	Azure Managed Redis
"Exact recall on a small vector set"	`flat` index (Cosmos) or `FLAT` (Redis)
"Millions of vectors, low latency"	diskANN (Cosmos) or HNSW (pgvector / Redis)

A.3 Which messaging service?

Signal	Answer
"Must be processed", "retry", "dead-letter", "ordering"	Service Bus
"Something happened", "react", "fan out to subscribers"	Event Grid
"Millions of telemetry events per second", "replay by offset"	Event Hubs
"Simplest possible queue, very large, cheap"	Storage queue

A.4 Which security control?

Signal	Answer
"No credentials in code or config"	Managed identity + RBAC
"Secret must be rotatable and audited"	Key Vault (+ rotation policy and Event Grid)
"Change a value without redeploying"	App Configuration (label per environment)
"Turn a feature on for 10% of users"	App Configuration feature flag with a filter
"Secret must not appear in an exported ARM template"	Key Vault reference in the app setting

A.5 Which telemetry answer?

Signal	Answer
"Where did the time go across services?"	Distributed trace, correlate on `operation_Id`
"Which dependency is slow?"	`dependencies` table, p95 by target
"Why did this one request fail?"	`exceptions` + union on `operation_Id`
"Are we being throttled?"	`dependencies` where `resultCode == "429"`
"Trace disappears after the queue"	Propagate `traceparent` in the message properties

Appendix B — Every Microsoft Learn link in this guide

Every link below already carries the Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Share any of them directly.

Resource	URL (includes Contributor ID)
Microsoft Certified: Azure AI Cloud Developer Associate	https://learn.microsoft.com/credentials/certifications/azure-ai-cloud-developer-associate/?wt.mc_id=studentamb_657938
official AI-200 study guide	https://learn.microsoft.com/credentials/certifications/resources/study-guides/ai-200?wt.mc_id=studentamb_657938
exam duration and experience	https://learn.microsoft.com/credentials/support/exam-duration-exam-experience?wt.mc_id=studentamb_657938
exam scoring and score reports	https://learn.microsoft.com/credentials/certifications/exam-scoring-reports?wt.mc_id=studentamb_657938
retake policy	https://learn.microsoft.com/credentials/support/retake-policy?wt.mc_id=studentamb_657938
renew your certification	https://learn.microsoft.com/credentials/certifications/renew-your-microsoft-certification?wt.mc_id=studentamb_657938
Request accommodations	https://learn.microsoft.com/credentials/certifications/request-accommodations?wt.mc_id=studentamb_657938
AI-200 exam page	https://learn.microsoft.com/credentials/certifications/exams/ai-200/?wt.mc_id=studentamb_657938
Implement container application hosting on Azure	https://learn.microsoft.com/training/paths/implement-container-app-hosting-azure/?wt.mc_id=studentamb_657938
Deploy and manage apps on Azure Container Apps	https://learn.microsoft.com/training/paths/deploy-manage-apps-azure-container-apps/?wt.mc_id=studentamb_657938
Deploy containers with AKS	https://learn.microsoft.com/training/paths/deploy-manage-containers-azure-kubernetes-service/?wt.mc_id=studentamb_657938
Develop AI solutions with Azure Cosmos DB for NoSQL	https://learn.microsoft.com/training/paths/develop-ai-solutions-azure-cosmos-db/?wt.mc_id=studentamb_657938
Build AI apps with Azure Database for PostgreSQL	https://learn.microsoft.com/training/paths/build-ai-apps-azure-database-postgresql/?wt.mc_id=studentamb_657938
Enhance AI solutions with Azure Managed Redis	https://learn.microsoft.com/training/paths/enhance-ai-solutions-azure-managed-redis/?wt.mc_id=studentamb_657938
Integrate backend services for AI solutions	https://learn.microsoft.com/training/paths/integrate-backend-services-ai-solutions/?wt.mc_id=studentamb_657938
Manage application secrets and configuration	https://learn.microsoft.com/training/paths/manage-app-secrets-configuration/?wt.mc_id=studentamb_657938
Observe and troubleshoot apps on Azure	https://learn.microsoft.com/training/paths/observe-troubleshoot-apps/?wt.mc_id=studentamb_657938
Azure free account	https://azure.microsoft.com/free/students?wt.mc_id=studentamb_657938
App Service	https://learn.microsoft.com/azure/app-service/overview?wt.mc_id=studentamb_657938
Container Apps	https://learn.microsoft.com/azure/container-apps/overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
AKS	https://learn.microsoft.com/azure/aks/what-is-aks?wt.mc_id=studentamb_657938
Service Bus	https://learn.microsoft.com/azure/service-bus-messaging/service-bus-messaging-overview?wt.mc_id=studentamb_657938
Event Grid	https://learn.microsoft.com/azure/event-grid/overview?wt.mc_id=studentamb_657938
Functions	https://learn.microsoft.com/azure/azure-functions/functions-overview?wt.mc_id=studentamb_657938
Cosmos DB	https://learn.microsoft.com/azure/cosmos-db/nosql/?wt.mc_id=studentamb_657938
PostgreSQL	https://learn.microsoft.com/azure/postgresql/flexible-server/overview?wt.mc_id=studentamb_657938
Managed Redis	https://learn.microsoft.com/azure/redis/overview?wt.mc_id=studentamb_657938
Azure OpenAI	https://learn.microsoft.com/azure/ai-services/openai/overview?wt.mc_id=studentamb_657938
Key Vault	https://learn.microsoft.com/azure/key-vault/general/overview?wt.mc_id=studentamb_657938
App Configuration	https://learn.microsoft.com/azure/azure-app-configuration/overview?wt.mc_id=studentamb_657938
Azure Monitor	https://learn.microsoft.com/azure/azure-monitor/app/app-insights-overview?wt.mc_id=studentamb_657938
managed identities for Azure resources	https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/overview?wt.mc_id=studentamb_657938
authentication overview for the Azure SDK for Python	https://learn.microsoft.com/azure/developer/python/sdk/authentication-overview?wt.mc_id=studentamb_657938
Cosmos DB vector search	https://learn.microsoft.com/azure/cosmos-db/nosql/vector-search?wt.mc_id=studentamb_657938
pgvector	https://learn.microsoft.com/azure/postgresql/flexible-server/how-to-use-pgvector?wt.mc_id=studentamb_657938
OpenTelemetry	https://learn.microsoft.com/azure/azure-monitor/app/opentelemetry-overview?wt.mc_id=studentamb_657938
Azure Container Registry (ACR)	https://learn.microsoft.com/azure/container-registry/container-registry-intro?wt.mc_id=studentamb_657938
registry authentication	https://learn.microsoft.com/azure/container-registry/container-registry-authentication?wt.mc_id=studentamb_657938
image tag best practices	https://learn.microsoft.com/azure/container-registry/container-registry-image-tag-version?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
ACR Tasks	https://learn.microsoft.com/azure/container-registry/container-registry-tasks-overview?wt.mc_id=studentamb_657938
run a custom container	https://learn.microsoft.com/azure/app-service/quickstart-custom-container?wt.mc_id=studentamb_657938
Key Vault references for App Service	https://learn.microsoft.com/azure/app-service/app-service-key-vault-references?wt.mc_id=studentamb_657938
deployment slots	https://learn.microsoft.com/azure/app-service/deploy-staging-slots?wt.mc_id=studentamb_657938
configure common settings	https://learn.microsoft.com/azure/app-service/configure-common?wt.mc_id=studentamb_657938
run sidecar-enabled AI applications on Azure App Service	https://learn.microsoft.com/training/modules/deploy-sidecar-azure-app-service/?wt.mc_id=studentamb_657938
registry concepts	https://learn.microsoft.com/azure/container-registry/container-registry-concepts?wt.mc_id=studentamb_657938
managed identity	https://learn.microsoft.com/azure/app-service/overview-managed-identity?wt.mc_id=studentamb_657938
Store and manage containers in ACR	https://learn.microsoft.com/training/modules/store-manage-containers-azure-container-registry/?wt.mc_id=studentamb_657938
Deploy containers to App Service	https://learn.microsoft.com/training/modules/deploy-containers-azure-app-service/?wt.mc_id=studentamb_657938
environment	https://learn.microsoft.com/azure/container-apps/environment?wt.mc_id=studentamb_657938
revision	https://learn.microsoft.com/azure/container-apps/revisions?wt.mc_id=studentamb_657938
KEDA	https://learn.microsoft.com/azure/container-apps/scale-app?wt.mc_id=studentamb_657938
dynamic sessions	https://learn.microsoft.com/azure/container-apps/sessions?wt.mc_id=studentamb_657938
serverless GPUs	https://learn.microsoft.com/azure/container-apps/gpu-serverless-overview?wt.mc_id=studentamb_657938
KEDA on AKS	https://learn.microsoft.com/azure/aks/keda-about?wt.mc_id=studentamb_657938
scaling concepts	https://learn.microsoft.com/azure/aks/concepts-scale?wt.mc_id=studentamb_657938
configure scaling	https://learn.microsoft.com/training/modules/configure-scaling-azure-kubernetes-service/?wt.mc_id=studentamb_657938
workload identity	https://learn.microsoft.com/azure/aks/workload-identity-overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Container insights	https://learn.microsoft.com/azure/aks/monitor-aks?wt.mc_id=studentamb_657938
AKS diagnostics	https://learn.microsoft.com/azure/aks/aks-diagnostics?wt.mc_id=studentamb_657938
AKS troubleshooting documentation	https://learn.microsoft.com/troubleshoot/azure/azure-kubernetes/welcome-azure-kubernetes?wt.mc_id=studentamb_657938
log monitoring	https://learn.microsoft.com/azure/container-apps/log-monitoring?wt.mc_id=studentamb_657938
observability	https://learn.microsoft.com/azure/container-apps/observability?wt.mc_id=studentamb_657938
managed identity	https://learn.microsoft.com/azure/container-apps/managed-identity?wt.mc_id=studentamb_657938
ingress	https://learn.microsoft.com/azure/container-apps/ingress-overview?wt.mc_id=studentamb_657938
clusters and workloads	https://learn.microsoft.com/azure/aks/concepts-clusters-workloads?wt.mc_id=studentamb_657938
networking concepts	https://learn.microsoft.com/azure/aks/concepts-network?wt.mc_id=studentamb_657938
Scale containers in Container Apps	https://learn.microsoft.com/training/modules/scale-containers-azure-container-apps/?wt.mc_id=studentamb_657938
Run AI-generated code in dynamic sessions	https://learn.microsoft.com/training/modules/dynamic-sessions-azure-container-apps/?wt.mc_id=studentamb_657938
hierarchical partition keys	https://learn.microsoft.com/azure/cosmos-db/partitioning-overview?wt.mc_id=studentamb_657938
indexing policies	https://learn.microsoft.com/azure/cosmos-db/index-policy?wt.mc_id=studentamb_657938
optimize cost of reads and writes	https://learn.microsoft.com/azure/cosmos-db/optimize-cost-reads-writes?wt.mc_id=studentamb_657938
consistency levels	https://learn.microsoft.com/azure/cosmos-db/consistency-levels?wt.mc_id=studentamb_657938
Azure Cosmos DB as a vector database	https://learn.microsoft.com/azure/cosmos-db/vector-database?wt.mc_id=studentamb_657938
change feed	https://learn.microsoft.com/azure/cosmos-db/change-feed?wt.mc_id=studentamb_657938
Change feed processor	https://learn.microsoft.com/azure/cosmos-db/nosql/change-feed-processor?wt.mc_id=studentamb_657938
getting started with queries	https://learn.microsoft.com/azure/cosmos-db/nosql/query/getting-started?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
request units	https://learn.microsoft.com/azure/cosmos-db/request-units?wt.mc_id=studentamb_657938
get started with Python	https://learn.microsoft.com/azure/cosmos-db/nosql/how-to-python-get-started?wt.mc_id=studentamb_657938
Implement vector search	https://learn.microsoft.com/training/modules/implement-vector-search-azure-cosmos-db/?wt.mc_id=studentamb_657938
Optimize query performance	https://learn.microsoft.com/training/modules/optimize-query-performance-azure-cosmos-db/?wt.mc_id=studentamb_657938
Handle events with the change feed	https://learn.microsoft.com/training/modules/handle-events-azure-functions-azure-cosmos-db-sql-api-change-feed/?wt.mc_id=studentamb_657938
generative AI with Azure Database for PostgreSQL	https://learn.microsoft.com/azure/postgresql/flexible-server/generative-ai-overview?wt.mc_id=studentamb_657938
PgBouncer	https://learn.microsoft.com/azure/postgresql/flexible-server/concepts-pgbouncer?wt.mc_id=studentamb_657938
compute options	https://learn.microsoft.com/azure/postgresql/flexible-server/concepts-compute?wt.mc_id=studentamb_657938
index tuning	https://learn.microsoft.com/azure/postgresql/flexible-server/concepts-index-tuning?wt.mc_id=studentamb_657938
Azure OpenAI integration	https://learn.microsoft.com/azure/postgresql/flexible-server/generative-ai-azure-openai?wt.mc_id=studentamb_657938
Enable semantic search	https://learn.microsoft.com/training/modules/enable-semantic-search-azure-database-postgresql/?wt.mc_id=studentamb_657938
Build RAG applications	https://learn.microsoft.com/training/modules/build-rag-applications-azure-database-postgresql/?wt.mc_id=studentamb_657938
Azure Cache for Redis overview	https://learn.microsoft.com/azure/azure-cache-for-redis/cache-overview?wt.mc_id=studentamb_657938
vector similarity tutorial	https://learn.microsoft.com/azure/redis/tutorial-vector-similarity?wt.mc_id=studentamb_657938
data operations	https://learn.microsoft.com/training/modules/implement-data-operations-azure-managed-redis/?wt.mc_id=studentamb_657938
vector storage	https://learn.microsoft.com/training/modules/implement-vector-storage-azure-managed-redis/?wt.mc_id=studentamb_657938
event messaging	https://learn.microsoft.com/training/modules/implement-event-messaging-azure-managed-redis/?wt.mc_id=studentamb_657938
dead-letter queues	https://learn.microsoft.com/azure/service-bus-messaging/service-bus-dead-letter-queues?wt.mc_id=studentamb_657938
event filtering	https://learn.microsoft.com/azure/event-grid/event-filtering?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
delivery and retry	https://learn.microsoft.com/azure/event-grid/delivery-and-retry?wt.mc_id=studentamb_657938
event schema	https://learn.microsoft.com/azure/event-grid/event-schema?wt.mc_id=studentamb_657938
custom events quickstart	https://learn.microsoft.com/azure/event-grid/custom-event-quickstart?wt.mc_id=studentamb_657938
queues, topics, subscriptions	https://learn.microsoft.com/azure/service-bus-messaging/service-bus-queues-topics-subscriptions?wt.mc_id=studentamb_657938
topic filters	https://learn.microsoft.com/azure/service-bus-messaging/topic-filters?wt.mc_id=studentamb_657938
message sessions	https://learn.microsoft.com/azure/service-bus-messaging/message-sessions?wt.mc_id=studentamb_657938
concepts	https://learn.microsoft.com/azure/event-grid/concepts?wt.mc_id=studentamb_657938
MQTT	https://learn.microsoft.com/azure/event-grid/mqtt-overview?wt.mc_id=studentamb_657938
Queue and process AI operations with Azure Service Bus	https://learn.microsoft.com/training/modules/queue-process-operations-service-bus/?wt.mc_id=studentamb_657938
Develop event-driven AI workflows with Event Grid	https://learn.microsoft.com/training/modules/event-driven-workflows-event-grid/?wt.mc_id=studentamb_657938
Azure OpenAI extension	https://learn.microsoft.com/azure/azure-functions/functions-bindings-openai?wt.mc_id=studentamb_657938
hosting and scale	https://learn.microsoft.com/azure/azure-functions/functions-scale?wt.mc_id=studentamb_657938
deployment technologies	https://learn.microsoft.com/azure/azure-functions/functions-deployment-technologies?wt.mc_id=studentamb_657938
Durable Functions	https://learn.microsoft.com/azure/azure-functions/durable/durable-functions-overview?wt.mc_id=studentamb_657938
triggers and bindings	https://learn.microsoft.com/azure/azure-functions/functions-triggers-bindings?wt.mc_id=studentamb_657938
Python developer guide	https://learn.microsoft.com/azure/azure-functions/functions-reference-python?wt.mc_id=studentamb_657938
local development	https://learn.microsoft.com/azure/azure-functions/functions-develop-local?wt.mc_id=studentamb_657938
Build serverless AI backends with Azure Functions	https://learn.microsoft.com/training/modules/build-backends-azure-functions/?wt.mc_id=studentamb_657938
Orchestrate durable AI workflows	https://learn.microsoft.com/training/modules/orchestrate-durable-functions/?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
secret rotation tutorial	https://learn.microsoft.com/azure/key-vault/secrets/tutorial-rotation?wt.mc_id=studentamb_657938
Key Vault developer's guide	https://learn.microsoft.com/azure/key-vault/general/developers-guide?wt.mc_id=studentamb_657938
feature management	https://learn.microsoft.com/azure/azure-app-configuration/concept-feature-management?wt.mc_id=studentamb_657938
about secrets	https://learn.microsoft.com/azure/key-vault/secrets/about-secrets?wt.mc_id=studentamb_657938
RBAC guide	https://learn.microsoft.com/azure/key-vault/general/rbac-guide?wt.mc_id=studentamb_657938
key-values	https://learn.microsoft.com/azure/azure-app-configuration/concept-key-value?wt.mc_id=studentamb_657938
best practices	https://learn.microsoft.com/azure/azure-app-configuration/howto-best-practices?wt.mc_id=studentamb_657938
Key Vault	https://learn.microsoft.com/training/modules/manage-app-secrets-key-vault/?wt.mc_id=studentamb_657938
App Configuration	https://learn.microsoft.com/training/modules/manage-app-settings-app-config/?wt.mc_id=studentamb_657938
Azure Monitor OpenTelemetry Distro	https://learn.microsoft.com/azure/azure-monitor/app/opentelemetry-enable?wt.mc_id=studentamb_657938
distributed tracing	https://learn.microsoft.com/azure/azure-monitor/app/distributed-trace-data?wt.mc_id=studentamb_657938
KQL	https://learn.microsoft.com/azure/azure-monitor/logs/log-query-overview?wt.mc_id=studentamb_657938
get started with queries	https://learn.microsoft.com/azure/azure-monitor/logs/get-started-queries?wt.mc_id=studentamb_657938
Log Analytics tutorial	https://learn.microsoft.com/azure/azure-monitor/logs/log-analytics-tutorial?wt.mc_id=studentamb_657938
Instrument an app with OpenTelemetry	https://learn.microsoft.com/training/modules/instrument-app-opentelemetry/?wt.mc_id=studentamb_657938
Analyze app telemetry with logs and metrics	https://learn.microsoft.com/training/modules/analyze-telemetry-logs-metrics/?wt.mc_id=studentamb_657938
Deploy containers to Azure Container Apps	https://learn.microsoft.com/training/modules/deploy-containers-azure-container-apps/?wt.mc_id=studentamb_657938
Manage containers in Azure Container Apps	https://learn.microsoft.com/training/modules/manage-containers-azure-container-apps/?wt.mc_id=studentamb_657938
Deploy an Azure Kubernetes Service cluster	https://learn.microsoft.com/training/modules/deploy-azure-kubernetes-service-cluster/?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Deploy applications to AKS	https://learn.microsoft.com/training/modules/deploy-applications-azure-kubernetes-service/?wt.mc_id=studentamb_657938
Build queries for Azure Cosmos DB for NoSQL	https://learn.microsoft.com/training/modules/build-query-azure-cosmos-db/?wt.mc_id=studentamb_657938
Get started with generative AI in PostgreSQL	https://learn.microsoft.com/training/modules/get-started-generative-ai-azure-database-postgresql/?wt.mc_id=studentamb_657938
AI-300	https://learn.microsoft.com/credentials/certifications/operationalizing-machine-learning-and-generative-ai-solutions/?wt.mc_id=studentamb_657938
Azure updates feed	https://azure.microsoft.com/updates/?wt.mc_id=studentamb_657938
Azure Architecture Center	https://learn.microsoft.com/azure/architecture/?wt.mc_id=studentamb_657938
Microsoft Q&A	https://learn.microsoft.com/answers/products/?wt.mc_id=studentamb_657938
Azure community support	https://azure.microsoft.com/support/community/?wt.mc_id=studentamb_657938